

CARTRIDGE & CLOUD · DOCUMENTACIÓN RC1

Cartridge & Cloud — Technical Design Document

Edición PDF con identidad visual unificada de VRM Games. El contenido procede íntegramente del archivo Markdown original.

PROYECTO

Cartridge & Cloud

ESTADO

Fuente vigente de diseño técnico

DOCUMENTO

03 / 03_Technical_Design_Document.md

AUTORÍA

VRM Games / Blas Luis Rocha González

VERSIÓN

1.0

FECHA DOCUMENTAL

2026-07-01

Control y metadatos del documento

Archivo fuente: 03_Technical_Design_Document.md
SHA-256 del Markdown: 66264703c5ff4959d03093690e9845a98ec0e5025e685b9a639ac8cbb616cd12
Tamaño del Markdown: 101,994 bytes
Conversión: representación visual completa; el Markdown original mantiene la autoridad sobre el texto fuente.

Front matter original

title	Cartridge & Cloud — Technical Design Document
author	VRM Games / Blas Luis Rocha González
date	2026-07-01
lang	es-ES
document_version	1.0
status	Fuente vigente de diseño técnico
project	Cartridge & Cloud
platform	PC / Steam
engine	Unity 6.3 LTS 6000.3.18f1
render_pipeline	URP 17.3.0
application_version_reference	0.0.17

Índice

Cartridge & Cloud — Technical Design Document

- 0. Propósito, alcance y autoridad
- 1. Baseline técnica consolidada
 - 1.1. Plataforma y stack
 - 1.2. Paquetes principales observados
 - 1.3. Dimensión de la instantánea técnica
 - 1.4. Estado funcional observado
 - 1.5. Escenas observadas
- 2. Principios arquitectónicos obligatorios
 - 2.1. Dependencias dirigidas hacia el núcleo
 - 2.2. Dominio antes que MonoBehaviour
 - 2.3. Validar antes de mutar
 - 2.4. Mutaciones atómicas e idempotencia
 - 2.5. IDs estables
 - 2.6. Escenas como composición, no como base de datos
 - 2.7. Autoría explícita del espacio
 - 2.8. Datos estáticos separados del estado runtime
 - 2.9. Determinismo donde aporta valor
 - 2.10. Resultados tipados
 - 2.11. Composición explícita
 - 2.12. Fallo temprano en authoring
 - 2.13. Patrones prohibidos
- 3. Capas, assemblies y reglas de dependencia
 - 3.1. Domain
 - 3.2. Application
 - 3.3. Infrastructure
 - 3.4. Presentation
 - 3.5. Runtime composition
 - 3.6. Editor
 - 3.7. Tests
 - 3.8. Assemblies observados
 - 3.9. Observaciones sobre dependencias actuales
 - 3.10. Criterios para crear un nuevo assembly
- 4. Composición, ciclo de vida y escenas
 - 4.1. ApplicationRoot
 - 4.2. Navegación
 - 4.3. Roles de escena
 - 4.4. Estado de la lista de build
 - 4.5. Shutdown
- 5. Identidad, datos y catálogos
 - 5.1. Familia de IDs
 - 5.2. Reglas de ID
 - 5.3. ScriptableObjects como datos de authoring
 - 5.4. Validación de catálogos
 - 5.5. RuntimeAssetRegistry
 - 5.6. Versionado de contenido
- 6. Input y control contextual
 - 6.1. Contextos
 - 6.2. Input Actions
 - 6.3. EventSystem y supresión de input de mundo
 - 6.4. Conflicto movimiento/placement
 - 6.5. Lectura por frame
 - 6.6. Rebinding y accesibilidad
- 7. Movimiento, cámara y navegación
 - 7.1. Movimiento por clic
 - 7.2. Reglas de destino
 - 7.3. Cámara orbital
 - 7.4. Navegación de agentes

- 7.5. Construcción y NavMesh
- 8. Grid, placement, ocupación y acceso
 - 8.1. Modelo lógico
 - 8.2. Rotación
 - 8.3. PlacementOccupancyRegistry
 - 8.4. Secuencia de placement
 - 8.5. Preview
 - 8.6. Validación de acceso
 - 8.7. StoreShellSpecification
 - 8.8. Persistencia de placement
- 9. Productos y catálogos comerciales
 - 9.1. ProductDefinition
 - 9.2. Registry
 - 9.3. Catálogo representativo
 - 9.4. Precio
 - 9.5. Compatibilidad visual
- 10. Inventario y transferencias
 - 10.1. Modelo
 - 10.2. Invariantes
 - 10.3. Transferencia atómica
 - 10.4. Capacidad
 - 10.5. Auditoría
- 11. Proveedores, pedidos, entregas y recepción
 - 11.1. SupplierCatalog
 - 11.2. PurchaseOrder
 - 11.3. Creación
 - 11.4. Delivery y ShipmentBox
 - 11.5. ReceivingService
 - 11.6. Economía de recepción
 - 11.7. Presentación
- 12. Displays, asignación y reposición
 - 12.1. DisplayDefinition e instancia
 - 12.2. Asignación
 - 12.3. Reposición
 - 12.4. Retorno
 - 12.5. Sincronización visual
- 13. Clientes, spawning, navegación y shopping
 - 13.1. CustomerProfile
 - 13.2. Estados del cliente
 - 13.3. Spawning
 - 13.4. Navegación
 - 13.5. Búsqueda y disponibilidad
 - 13.6. Reserva lógica
 - 13.7. Carrito y sesión
 - 13.8. Paciencia y abandono
 - 13.9. Consistencia global
- 14. Cola, estación y checkout
 - 14.1. Cola FIFO
 - 14.2. Estación
 - 14.3. Preflight de checkout
 - 14.4. Commit
 - 14.5. Cancelación
 - 14.6. Idempotencia
- 15. Ciclo diario, cierre y economía
 - 15.1. Estados de jornada
 - 15.2. BeforeOpen
 - 15.3. Open
 - 15.4. Closing
 - 15.5. Closed
 - 15.6. Money
 - 15.7. Ledger
 - 15.8. Resultados diarios
 - 15.9. Fiscalidad
- 16. Persistencia, slots, backup y recuperación
 - 16.1. Dos niveles observados
 - 16.2. Snapshot integrado

- 16.3. Validación cruzada
- 16.4. Escritura atómica
- 16.5. Estados de repository
- 16.6. Backup y recovery
- 16.7. Migraciones
- 16.8. Autosave
- 16.9. Rutas y archivos
- 17. UI, tutorial, accesibilidad y proyecciones
 - 17.1. Separación
 - 17.2. Capas UI
 - 17.3. Paneles de gestión
 - 17.4. StoreUiProjectionService
 - 17.5. Tutorial
 - 17.6. Accesibilidad
 - 17.7. MainMenu y slots
- 18. StoreInitial y assets representativos
 - 18.1. Problema de la solución procedural
 - 18.2. Jerarquía objetivo
 - 18.3. StoreInitialEnvironment.prefab
 - 18.4. StoreInitialSceneContext
 - 18.5. Secuencia runtime objetivo
 - 18.6. Mobiliario inicial y dinámico
 - 18.7. Puerta
 - 18.8. Factories representativas
 - 18.9. Migración segura
 - 18.10. Criterios de ausencia de duplicados
- 19. Herramientas de Editor y authoring
 - 19.1. Instaladores idempotentes
 - 19.2. Migraciones de organización
 - 19.3. Reparación de fixtures
 - 19.4. Validadores
 - 19.5. Límites
- 20. Estrategia de pruebas
 - 20.1. Pirámide
 - 20.2. Volumen observado
 - 20.3. Convenciones
 - 20.4. Cobertura obligatoria por sistema
 - 20.5. Golden Path
 - 20.6. Datos de prueba
 - 20.7. Tests frágiles
- 21. Logging, diagnóstico y manejo de errores
 - 21.1. Categorías
 - 21.2. Niveles
 - 21.3. Contexto mínimo
 - 21.4. Errores recuperables
 - 21.5. Errores no recuperables
 - 21.6. TechnicalScenarioRunner
- 22. Rendimiento y memoria
 - 22.1. Presupuestos iniciales
 - 22.2. Reglas
 - 22.3. Perfilado
 - 22.4. Escalabilidad
- 23. Build, versión y distribución interna
 - 23.1. Plataforma
 - 23.2. Preflight de build
 - 23.3. Build de desarrollo
 - 23.4. Validación externa al Editor
 - 23.5. Scene list
 - 23.6. Versionado
 - 23.7. Evidencias
- 24. Seguridad de datos e integridad
- 25. Arquitectura de sistemas futuros
 - 25.1. Regla general
 - 25.2. Empleados — DIFERIDO
 - 25.3. Investigación — DIFERIDO
 - 25.4. Puestos informáticos — DIFERIDO

- 25.5. Comercio online — MEDIO PLAZO
- 25.6. Publishing y desarrollo — VISIÓN TARDÍA
- 25.7. Plataforma e infraestructura — VISIÓN TARDÍA
- 26. Deuda técnica y migraciones conocidas
 - 26.1. StoreInitial fuera de build
 - 26.2. Builders procedurales
 - 26.3. Sprint16Phase1RuntimeRoot global
 - 26.4. Inyección por escaneo de escena
 - 26.5. Dependencia Infrastructure.InputSystem → Presentation
 - 26.6. Save skeleton e integrated save
 - 26.7. Acciones de plantilla no usadas
 - 26.8. Duplicación de relojes UTC
 - 26.9. Store y StoreInitial similares
 - 26.10. TechnicalScenarioRunners
 - 26.11. Priorización
- 27. ADR, control de cambios y revisión técnica
 - 27.1. Cuándo crear ADR
 - 27.2. Contenido mínimo
 - 27.3. Revisión de código
 - 27.4. Definition of Done técnica de una feature
- 28. Criterio de aceptación del TDD
 - 28.1. Gate técnico de Sprint 16
 - 28.2. Gate técnico de Sprint 17
 - 28.3. Condición para ampliar arquitectura
- 29. Comunicación entre sistemas, eventos y proyecciones
 - 29.1. Regla de comunicación
 - 29.2. Comandos
 - 29.3. Eventos
 - 29.4. Eventos versus coordinación transaccional
 - 29.5. Proyecciones
 - 29.6. Frecuencia de actualización
- 30. Tiempo, concurrencia y orden de ejecución
 - 30.1. Hilo principal
 - 30.2. Operaciones asíncronas
 - 30.3. Reentrancia
 - 30.4. Orden de inicialización
 - 30.5. Pausa
 - 30.6. Reloj de juego y reloj UTC
- 31. Localización, audio, VFX y feedback técnico
 - 31.1. Localización
 - 31.2. Audio
 - 31.3. VFX
 - 31.4. Feedback
 - 31.5. Accesibilidad del feedback
- 32. Configuración, balance y entornos
 - 32.1. Settings assets
 - 32.2. Separación de configuración
 - 32.3. Defaults
 - 32.4. Configuración por entorno
 - 32.5. Balance
- 33. Matriz técnica de propiedad de estado
- 34. Contratos de restauración de estado
 - 34.1. Principio
 - 34.2. Fases
 - 34.3. Restauración transaccional
 - 34.4. Reconciliación de escena
 - 34.5. Postcondiciones
- 35. Checklist de implementación por subsistema
 - 35.1. Nuevo tipo de dominio
 - 35.2. Nuevo caso de uso
 - 35.3. Nuevo MonoBehaviour
 - 35.4. Nuevo ScriptableObject
 - 35.5. Cambio de escena
 - 35.6. Cambio de persistencia
 - 35.7. Cambio de build o paquete
- Anexo A. Assembly map observado

Anexo B. Inventario de clases técnicas clave

Anexo C. Distribución de código

Anexo D. Escenas y build

 D.1. Escenas presentes

 D.2. Orden observado

Anexo E. Trazabilidad de fuentes

 E.1. Resoluciones principales

Anexo F. Glosario técnico

Cartridge & Cloud — Technical Design Document

0. Propósito, alcance y autoridad

Este documento define la arquitectura técnica vigente de *Cartridge & Cloud*, las responsabilidades de cada capa, las reglas de dependencia, los contratos entre sistemas, las invariantes que deben protegerse, la composición de escenas, la persistencia, la estrategia de pruebas, los requisitos de rendimiento, el proceso de build y las restricciones que deben respetar los sistemas futuros.

No es una enumeración de clases ni una copia del código. Su función es permitir que una persona pueda comprender:

- qué arquitectura existe;
- qué parte está implementada;
- qué parte es objetivo aprobado;
- cómo debe extenderse cada subsistema;
- qué dependencias están permitidas;
- qué estados e invariantes son obligatorios;
- qué deuda técnica es transitoria;
- qué evidencias se necesitan antes de cerrar un cambio.

El TDD convierte los requisitos de diseño en una estructura implementable y verificable. No puede redefinir la fantasía, el alcance o las reglas funcionales fijadas por documentos de mayor autoridad.

0.1. Jerarquía de autoridad

La jerarquía aplicada es:

1. `00_Enfoque_y_Alcance.md` para visión, pilares, límites y sistemas diferidos.
2. `01_Game_Design_Document.md` para reglas funcionales y comportamiento esperado.
3. `02_Vertical_Slice_Specification.md` para alcance obligatorio, criterios de aceptación, Golden Path, gates y Definition of Done.
4. TDD v0.6 para arquitectura actual de escena, assets representativos y migración de `StoreInitial`.
5. TDD v0.5 para la arquitectura implementada hasta Sprint 5, grid, acceso, input y composición inicial.
6. TDD v0.4 para la arquitectura extensa por capas, contextos, servicios, persistencia, rendimiento y riesgos.

7. TDD v0.3 como intención técnica histórica cuando no existe una formulación posterior.
8. La instantánea de código suministrada, como evidencia de implementación real en la fecha de consolidación.

Cuando exista una discrepancia:

- la dirección aprobada prevalece sobre la implementación accidental;
- el código prevalece únicamente para describir el estado observado, no para legitimar una desviación no aprobada;
- las decisiones más recientes sustituyen las antiguas;
- una dependencia o patrón presente en código puede clasificarse como deuda, no como norma;
- ningún sistema futuro se introduce por estar mencionado en TDD históricos;
- toda resolución estructural debe quedar registrada en un ADR o registro equivalente.

0.2. Estados técnicos utilizados

Estado	Significado
VIGENTE	Regla técnica obligatoria para nuevos cambios.
IMPLEMENTADO	Existe código integrado y evidencia en la instantánea suministrada.
OBJETIVO APROBADO	Diseño técnico acordado, todavía no cerrado en integración.
TRANSITORIO	Solución mantenida durante una migración controlada.
DEUDA CONOCIDA	Desviación aceptada temporalmente y con salida definida.
HIPÓTESIS	Alternativa pendiente de spike o validación.
PROHIBIDO	Patrón que no debe introducirse.
DIFERIDO	Arquitectura futura que no debe implementarse en el gate actual.

0.3. Regla de interpretación

La presencia de una arquitectura futura no autoriza su desarrollo inmediato. Los capítulos sobre empleados, investigación, puestos informáticos, comercio online, publishing, desarrollo interno, plataforma o infraestructura definen límites de compatibilidad; no son backlog del vertical slice.

Las cifras de rendimiento, capacidad o tiempos son presupuestos técnicos iniciales. Deben medirse y revisarse con evidencia, no relajarse por intuición.

1. Baseline técnica consolidada

1.1. Plataforma y stack

La plataforma inicial es Windows x64 para distribución futura en PC/Steam. La baseline suministrada utiliza:

- Unity 6000.3.18f1;
- Universal Render Pipeline 17.3.0;
- C#;
- Unity Input System;
- AI Navigation/NavMesh;
- Canvas/uGUI y TextMeshPro;
- Unity Localization;
- Unity Test Framework;
- serialización JSON versionada para persistencia local.

El proyecto no debe actualizar Unity, URP, Input System, AI Navigation ni Test Framework como parte incidental de una feature. Toda actualización de paquete constituye un cambio técnico independiente y debe incluir:

1. motivación;
2. revisión de changelog;
3. copia o rama segura;
4. compilación limpia;
5. ejecución de suites;
6. smoke de escenas;
7. Golden Path;
8. build Windows x64;
9. registro de problemas y rollback.

1.2. Paquetes principales observados

Paquete	Versión	Responsabilidad
com.unity.ai.navigation	2.0.13	Navegación y NavMesh.

Paquete	Versión	Responsabilidad
com.unity.ide.visualstudio	2.0.26	Integración con Visual Studio.
com.unity.inputsystem	1.19.0	Input Actions, mapas y dispositivos.
com.unity.localization	1.5.12	Localización ES/EN y tablas.
com.unity.render-pipelines.universal	17.3.0	Render URP.
com.unity.test-framework	1.6.0	Pruebas EditMode y PlayMode.
com.unity.ugui	2.0.0	Canvas/uGUI y EventSystem.

1.3. Dimensión de la instantánea técnica

La instantánea suministrada contiene:

- 457 archivos C#;
- 300 scripts bajo `Assets/_Project/Scripts`;
- 14 scripts de Editor;
- 143 scripts de pruebas;
- 12 archivos `.asmdef`;
- 5 escenas;
- 64 prefabs;
- 54 assets serializados;
- aproximadamente 1.270 métodos marcados con `[Test]` o `[UnityTest]`.

Estas cifras describen la entrega analizada. No deben convertirse en objetivos de volumen. Una solución con menos clases puede ser mejor si conserva responsabilidades, pruebas e invariantes.

1.4. Estado funcional observado

Área	Estado	Evidencia principal
Arranque y navegación	IMPLEMENTADO	<code>ApplicationRoot</code> , <code>SceneTransitionGate</code> , controladores de escena
Sesión y slots	IMPLEMENTADO	<code>GameSessionService</code> , repositorios JSON y UI de slots
Input contextual	IMPLEMENTADO	<code>InputContextService</code> , router y drivers Input System
Movimiento y cámara	IMPLEMENTADO	calculadores puros y componentes de presentación

Área	Estado	Evidencia principal
Grid y placement	IMPLEMENTADO	tipos de grid, preview, ocupación, acceso y runtime controller
Productos e inventario	IMPLEMENTADO	definiciones, contenedores, stacks y transferencias
Proveedores y pedidos	IMPLEMENTADO	catálogos, órdenes y transiciones
Entregas y recepción	IMPLEMENTADO	deliveries, boxes y receiving service
Displays y reposición	IMPLEMENTADO	asignación, capacidad, reposición y retorno
Cientes y shopping	IMPLEMENTADO	perfiles, spawning, estados, reservas, carritos y flujo
Cola y checkout	IMPLEMENTADO	cola FIFO, estación, cancelación y transacción
Día y cierre	IMPLEMENTADO	estados de jornada, drain y coordinator
Economía	IMPLEMENTADO	Money, ledger, ingresos, costes y resultados
Persistencia integrada	IMPLEMENTADO	snapshot, codec, repository, backup y restore
UI, tutorial y accesibilidad	IMPLEMENTADO	HUD, Operations, navegación, tutorial y settings
Presentación representativa	TRANSITORIO / EN CURSO	runtime root de Sprint 16, catálogos y builders
<code>StoreInitial</code> autorada	OBJETIVO APROBADO	escena existente, pendiente de composición final y gate
Build externa	PENDIENTE	<code>StoreInitial</code> aún no figura en la lista de escenas habilitadas

1.5. Escenas observadas

Escena	Función vigente	Estado de build
<code>Bootstrap.unity</code>	Crea o contiene <code>ApplicationRoot</code> ; redirige a menú	Incluida
<code>MainMenu.unity</code>	Selección de slot, nueva partida, carga, opciones y salida	Incluida
<code>Store.unity</code>	Escena histórica de integración jugable	Incluida
<code>TestLab.unity</code>	Regresión aislada y escenarios técnicos	Incluida
<code>StoreInitial.unity</code>	Nueva tienda representativa autorada	Existe, no incluida todavía

`StoreInitial` no debe sustituir a `Store` en la lista de build hasta superar los criterios de migración definidos en este TDD y en `02_Vertical_Slice_Specification.md`.

2. Principios arquitectónicos obligatorios

2.1. Dependencias dirigidas hacia el núcleo

La dirección conceptual es:

```
Domain
↑
Application
↑
Infrastructure / Presentation
↑
Runtime Composition / Scenes / Editor
```

`Domain` no depende de Unity. `Application` depende de `Domain`. Los adaptadores Unity implementan puertos definidos en capas interiores. La composición conoce implementaciones concretas; el dominio no conoce escenas, prefabs, GameObjects, archivos o dispositivos.

2.2. Dominio antes que MonoBehaviour

Una regla que pueda expresarse sin escena debe residir en `Domain` o `Application`:

- IDs y value objects;
- estados y transiciones;
- validación de cantidades;
- capacidad;
- reservas;
- colas;
- precios;
- transacciones;
- snapshots;
- resultados tipados;
- políticas deterministas.

`MonoBehaviour` debe encargarse de ciclo Unity, referencias serializadas, input, vistas, animación, física, NavMesh o composición. No debe convertirse en fuente autoritativa de una regla que pueda probarse sin PlayMode.

2.3. Validar antes de mutar

Toda operación susceptible de fallo sigue el patrón:

```
entrada
→ normalización
→ precondiciones
→ resolución de referencias
→ validación completa
→ preparación de cambios
→ commit atómico
→ evento / proyección / feedback
```

Una validación fallida no puede dejar cambios parciales. Cuando una operación coordina varios agregados, debe usar preflight y una secuencia de commit que permita abortar o restaurar de forma segura.

2.4. Mutaciones atómicas e idempotencia

Son obligatorias en:

- transferencias de inventario;
- recepción de cajas;
- reservas de shopping;
- consumo de reservas;
- checkout;
- movimientos económicos;
- cierre de día;
- guardado;
- migraciones de escena o assets.

Las operaciones externas o repetibles deben usar IDs estables o claves de posting que impidan procesar dos veces la misma entrega, venta, asiento o transición.

2.5. IDs estables

Los objetos persistentes y las relaciones entre contextos se identifican mediante IDs explícitos, no por:

- nombre de GameObject;

- índice de hijo;
- posición en jerarquía;
- referencia directa a escena;
- nombre de archivo visual;
- instancia runtime de Unity.

Los IDs deben:

- ser no vacíos;
- usar comparación ordinal;
- mantener formato estable;
- validarse al cargar;
- permanecer independientes del asset visual;
- evitar regeneración accidental durante una migración.

2.6. Escenas como composición, no como base de datos

La escena define:

- referencias espaciales;
- objetos visuales;
- colliders;
- iluminación;
- anchors;
- roots funcionales;
- componentes de bridge;
- contextos serializados.

La escena no debe ser la única fuente de verdad para:

- dinero;
- inventario;
- órdenes;
- estado de cliente;
- transacciones;

- día;
- progresión;
- datos de producto;
- reglas de capacidad.

2.7. Autoría explícita del espacio

La arquitectura de `StoreInitial` se autora manualmente. El runtime no debe deducir paredes, puertas, zonas o mobiliario estático a partir de nombres, bounds, escalas heredadas o ejes de un FBX.

El runtime debe registrar y operar un entorno aprobado. Puede instanciar elementos dinámicos, pero no reconstruir la arquitectura inicial.

2.8. Datos estáticos separados del estado runtime

Los datos estáticos incluyen:

- definiciones de producto;
- proveedores y ofertas;
- perfiles de cliente;
- displays;
- settings;
- paletas;
- catálogos visuales;
- clips de audio;
- balance.

Deben vivir en assets de configuración o estructuras inmutables. El estado runtime incluye cantidades, estados, reservas, posiciones, dinero y progreso. El save contiene IDs y valores, no referencias a `ScriptableObject`.

2.9. Determinismo donde aporta valor

El dominio debe aceptar relojes, aleatoriedad o generadores de IDs controlables cuando el resultado afecte a pruebas, persistencia o reproducción de fallos. Las pruebas no deben depender del reloj del sistema ni de `UnityEngine.Random` salvo que el objetivo sea probar el adaptador.

2.10. Resultados tipados

Las operaciones esperablemente fallables devuelven un resultado con:

- éxito o fallo;
- código o enum de causa;
- datos del resultado;
- mensaje localizable en la capa apropiada;
- ausencia de mutación en fallo.

Las excepciones se reservan para contratos rotos, argumentos de programación inválidos, corrupción o estados imposibles, no para flujo normal de usuario.

2.11. Composición explícita

Los servicios se construyen en roots o contextos de composición. No deben localizarse mediante búsquedas globales repetidas, `FindObjectOfType`, strings o singletons arbitrarios.

`ApplicationRoot.Instance` y el root transitorio de Sprint 16 son excepciones controladas, no precedentes para crear más singletons.

2.12. Fallo temprano en authoring

Los assets y escenas deben validarse en Editor o durante build cuando sea posible. Es preferible bloquear una build por:

- ID duplicado;
- referencia nula;
- catálogo incompleto;
- escena ausente;
- anchor obligatorio no asignado;
- prefab incompatible;
- schema desconocido;

que descubrir el problema durante una partida.

2.13. Patrones prohibidos

No se permite introducir:

- `GameManager` monolítico con todos los sistemas;
- estado autoritativo distribuido entre vistas;
- acceso global implícito a servicios;
- dependencias de `Domain` hacia Unity;
- guardado de referencias Unity;
- dinero en `float`;
- cantidades negativas toleradas;
- mutación durante una consulta o validación;
- búsquedas por nombre como contrato funcional;
- lógica de negocio en `Update` de cada entidad;
- instanciación masiva sin pooling cuando sea recurrente;
- regeneración de arquitectura en cada carga;
- borrado de fallbacks antes de completar la migración;
- excepciones capturadas y silenciadas;
- código de Editor incluido en runtime;
- tests que dependen del orden de ejecución de otros tests;
- nuevas features avanzadas antes de superar el gate vigente.

3. Capas, assemblies y reglas de dependencia

3.1. Domain

Responsabilidad:

- entidades, agregados y value objects;
- IDs;
- estados y transiciones;
- invariantes;

- registros persistentes;
- reglas puras;
- resultados de mutación de dominio.

No puede referenciar:

- `UnityEngine;`
- escenas;
- input;
- archivos;
- JSON concreto;
- UI;
- audio;
- prefabs;
- `MonoBehaviour;`
- servicios de plataforma.

La capa debe poder compilar y probarse como C# puro.

3.2. Application

Responsabilidad:

- casos de uso;
- coordinación de agregados;
- servicios deterministas;
- puertos e interfaces;
- políticas de input abstractas;
- captura y restauración de estado;
- resultados de operación;
- proyecciones de lectura sin dependencia visual.

Puede depender de `Domain`. No debe depender de escenas, componentes concretos o assets.

3.3. Infrastructure

Responsabilidad:

- repositorios JSON;
- filesystem;
- reloj UTC del sistema;
- Unity Input System;
- navegación de escenas;
- composición persistente;
- catálogos `ScriptableObject`;
- adaptadores de runtime;
- authoring assets;
- codecs;
- settings;
- bridges técnicos.

Infrastructure implementa puertos de Application. No debe absorber reglas de negocio.

3.4. Presentation

Responsabilidad:

- controladores de escena;
- cámara y vistas;
- movimiento visual;
- placement ghost;
- interacción con GameObjects;
- feedback y presentación;
- conversión entre mundo y modelos de aplicación cuando sea puramente visual.

Presentation no debe escribir directamente en persistencia ni resolver reglas económicas.

3.5. Runtime composition

El assembly `Runtime.VerticalSlicePhase1` puede conocer Domain, Application, Infrastructure y Presentation porque su función es componer el vertical slice. Esa libertad no autoriza a mover lógica de negocio al root runtime.

Debe limitarse a:

- resolver referencias;
- construir servicios;
- registrar vistas;
- sincronizar modelos y presentación;
- coordinar inicialización y shutdown;
- enrutar feedback.

3.6. Editor

Responsabilidad:

- instaladores idempotentes;
- migraciones de escenas y prefabs;
- validación de catálogos;
- reparación de referencias;
- generación controlada de fixtures;
- menús y comandos de authoring;
- comprobaciones de build.

Un script de Editor puede modificar assets de forma explícita, pero debe registrar qué cambió, poder ejecutarse más de una vez sin duplicar objetos y fallar con un diagnóstico accionable.

3.7. Tests

Se mantienen assemblies separados para:

- EditMode general;
- PlayMode general;
- EditMode de Input System;
- PlayMode de Input System.

Los tests pueden conocer las capas necesarias, pero no deben introducir dependencias en runtime.

3.8. Assemblies observados

Assembly	Ruta	Referencias	Plataforma	Auto referenc ed
VRMGames.CartridgeAndCloud.Editor.ProjectOrganization	Editor/ProjectOrganization/VRMGames.CartridgeAndCloud.Editor.ProjectOrganization.asmdef	VRMGames.CartridgeAndCloud.Domain , VRMGames.CartridgeAndCloud.Infrastructure , VRMGames.CartridgeAndCloud.Presentation	Editor	false
VRMGames.CartridgeAndCloud.Editor	Editor/VRMGames.CartridgeAndCloud.Editor.asmdef	VRMGames.CartridgeAndCloud.Application , VRMGames.CartridgeAndCloud.Presentation , VRMGames.CartridgeAndCloud.Infrastructure.InputSystem , Unity.InputSystem	Editor	false
VRMGames.CartridgeAndCloud.Application	Scripts/Application/VRMGames.CartridgeAndCloud.Application.asmdef	VRMGames.CartridgeAndCloud.Domain	Runtime/Editor según referencia	false
VRMGames.CartridgeAndCloud.Domain	Scripts/Domain/VRMGames.CartridgeAndCloud.Domain.asmdef	Ninguna	Runtime/Editor según referencia	false
VRMGames.CartridgeAndCloud.Infrastructure.InputSystem	Scripts/Infrastructure/InputSystem/VRMGames.CartridgeAndCloud.Infrastructure.InputSystem.asmdef	VRMGames.CartridgeAndCloud.Application , VRMGames.CartridgeAndCloud.Infrastructure , VRMGames.CartridgeAndCloud.Presentation , Unity.InputSystem	Runtime/Editor según referencia	false
VRMGames.CartridgeAndCloud.Infrastructure	Scripts/Infrastructure/VRMGames.CartridgeAndCloud.Infrastructure.asmdef	VRMGames.CartridgeAndCloud.Domain , VRMGames.CartridgeAndCloud.Application	Runtime/Editor según referencia	false
VRMGames.CartridgeAndCloud.Presentation	Scripts/Presentation/VRMGames.CartridgeAndCloud.Presentation.asmdef	VRMGames.CartridgeAndCloud.Domain , VRMGames.CartridgeAndCloud.Application	Runtime/Editor según referencia	false
VRMGames.CartridgeAndCloud.Runtime.VerticalSlicePhase1	Scripts/Runtime/VerticalSlicePhase1/VRMGames.CartridgeAndCloud.Runtime.VerticalSlicePhase1.asmdef	VRMGames.CartridgeAndCloud.Domain , VRMGames.CartridgeAndCloud.Application , VRMGames.CartridgeAndCloud.Infrastructure , VRMGames.CartridgeAndCloud.Presentation	Runtime/Editor según referencia	true
VRMGames.CartridgeAndCloud.Tests.EditMode	Tests/EditMode/VRMGames.CartridgeAndCloud.Tests.EditMode.asmdef	VRMGames.CartridgeAndCloud.Domain , VRMGames.CartridgeAndCloud.Application , VRMGames.CartridgeAndCloud.Infrastructure , VRMGames.CartridgeAndCloud.Presentation , UnityEngine.TestRunner , UnityEditor.TestRunner	Editor	false
VRMGames.CartridgeAndCloud.InputSystem.Tests.EditMode	Tests/InputSystem/EditMode/VRMGames.CartridgeAndCloud.InputSystem.Tests.EditMode.asmdef	VRMGames.CartridgeAndCloud.Application , VRMGames.CartridgeAndCloud.Infrastructure.InputSystem , Unity.InputSystem , UnityEngine.TestRunner , UnityEditor.TestRunner	Editor	false
VRMGames.CartridgeAndCloud.InputSystem.Tests.PlayMode	Tests/InputSystem/PlayMode/VRMGames.CartridgeAndCloud.InputSystem.Tests.PlayMode.asmdef	VRMGames.CartridgeAndCloud.Application , VRMGames.CartridgeAndCloud.Infrastructure , VRMGames.CartridgeAndCloud.Infrastructure.InputSystem , VRMGames.CartridgeAndCloud.Presentation , UnityEngine.TestRunner	Runtime/Editor según referencia	false
VRMGames.CartridgeAndCloud.Tests.PlayMode	Tests/PlayMode/VRMGames.CartridgeAndCloud.Tests.PlayMode.asmdef	VRMGames.CartridgeAndCloud.Domain , VRMGames.CartridgeAndCloud.Application , VRMGames.CartridgeAndCloud.Infrastructure , VRMGames.CartridgeAndCloud.Presentation , VRMGames.CartridgeAndCloud.Runtime.VerticalSlicePhase1 , UnityEngine.TestRunner	Runtime/Editor según referencia	false

3.9. Observaciones sobre dependencias actuales

La topología actual contiene dependencias pragmáticas que deben vigilarse:

- `Infrastructure.InputSystem` referencia `Presentation`. La dirección ideal sería que el adaptador dependiera de contratos de `Application` y que la composición conectara las vistas. No debe ampliarse esa referencia sin una revisión.
- `Runtime.VerticalSlicePhase1` referencia todas las capas. Es aceptable únicamente como composition root del slice.
- `Editor.ProjectOrganization` referencia runtime para inspeccionar y reparar assets. Debe permanecer Editor-only.
- `Presentation` referencia Domain y Application, pero no Infrastructure general. Esta separación debe conservarse.
- Todos los assemblies principales tienen `autoReferenced: false` salvo el runtime específico del vertical slice. La incorporación de un nuevo assembly debe justificar su política de auto-referencia.

3.10. Criterios para crear un nuevo assembly

Solo se crea cuando:

- existe una frontera estable;
- reduce tiempos o dependencias;
- necesita plataforma específica;
- separa código de Editor;
- requiere paquetes externos aislados;
- mejora testabilidad o build.

No se crea un assembly por cada carpeta o feature pequeña. Cada assembly añade coste de referencias, compilación, mantenimiento y diagnóstico.

4. Composición, ciclo de vida y escenas

4.1. ApplicationRoot

`ApplicationRoot` es el root persistente de la aplicación. En la instantánea observada:

- aplica `DontDestroyOnLoad`;
- impide duplicados mediante `Instance`;
- crea `GameSessionService`;
- crea `InputContextService`;
- implementa `ISceneNavigator`;
- mantiene un `SceneTransitionGate`;
- escucha `SceneManager.sceneLoaded`;
- aplica contexto de input por escena;
- inyecta consumidores encontrados en los roots de la escena;
- redirige `Bootstrap` hacia `MainMenu`.

La responsabilidad permitida de `ApplicationRoot` es composición global mínima. No debe incorporar inventario, clientes, economía, UI de tienda o lógica del día.

4.1.1. Invariantes

- solo existe una instancia;
- una transición concurrente se rechaza;
- una escena desconocida no se carga;
- solicitar la escena activa no inicia otra carga;
- los servicios globales existen antes de inyectar consumidores;
- la suscripción a `sceneLoaded` se elimina al destruir el root;
- una escena no debe depender de que el orden de roots sea específico.

4.1.2. Deuda transitoria

La inyección por escaneo de todos los `MonoBehaviour` de la escena es válida para la fundación, pero no debe crecer indefinidamente. La arquitectura objetivo usa contextos explícitos por escena y composición directa. La migración debe mantener compatibilidad hasta cubrir:

- navegación;
- sesión;
- input;
- UI;
- StoreInitial;
- tests de carga.

4.2. Navegación

`SceneTransitionGate` protege contra solicitudes simultáneas. Las rutas vigentes son:

```
Bootstrap → MainMenu  
MainMenu → Store o StoreInitial  
Store / StoreInitial → MainMenu  
TestLab → uso técnico
```

Una transición debe:

1. validar `SceneId`;
2. comprobar escena activa;
3. adquirir el gate;

4. iniciar carga asíncrona;
5. mantener input bloqueado o contextual;
6. esperar `isDone`;
7. liberar el gate;
8. aplicar contexto e inyección;
9. registrar éxito o fallo.

Si `LoadSceneAsync` no puede iniciarse, el gate debe liberarse y generarse un error visible.

4.3. Roles de escena

Bootstrap

Contiene la composición global mínima. No incluye contenido jugable.

MainMenu

Presenta slots, nueva partida, carga, opciones y salida. No contiene una copia del estado de juego; consulta servicios y proyecciones.

Store

Escena histórica y fallback funcional. Permanece hasta que `StoreInitial` supere regresión, Golden Path y build.

StoreInitial

Escena representativa objetivo. Debe contener arquitectura y referencias explícitas, no managers persistentes duplicados.

TestLab

Escena de laboratorio para probar placement, movimiento, cámara, visuales, clientes, UI y otros componentes sin cargar toda la tienda. No debe convertirse en una segunda aplicación.

4.4. Estado de la lista de build

La lista habilitada observada es:

```
0 Bootstrap
1 MainMenu
```

```
2 Store  
3 TestLab
```

`StoreInitial` existe en Assets, pero no está habilitada. Esto es coherente con una migración en curso. El gate para añadirla exige:

- contexto completo;
- ausencia de duplicados;
- references validadas;
- compatibilidad de save/load;
- input correcto;
- Golden Path;
- pruebas PlayMode;
- build Windows x64;
- `Player.log` sin errores críticos.

4.5. Shutdown

Los roots persistentes y services que se suscriben a eventos deben liberar:

- eventos de `SceneManager`;
- callbacks de Input System;
- coroutines;
- referencias a vistas destruidas;
- pools y objetos runtime;
- handles de archivo;
- tareas asíncronas.

No debe quedar un root de Sprint 16 activo después de abandonar la escena si sus vistas o servicios son específicos de la tienda.

5. Identidad, datos y catálogos

5.1. Familia de IDs

El proyecto usa wrappers específicos como:

- `StableId`;
- `SaveSlotId`;
- `ProductDefinitionId`;
- `InventoryContainerId`;
- `PurchaseOrderId`;
- `DeliveryId`;
- `DisplayInstanceId`;
- `CustomerInstanceId`;
- IDs de carrito, reserva, cola, transacción y anchors.

Cada tipo evita mezclar identificadores semánticamente distintos. No deben sustituirse por strings desnudos en APIs nuevas salvo en DTOs o serialización.

5.2. Reglas de ID

- construcción validada;
- valor no vacío;
- comparación ordinal;
- igualdad por valor;
- representación serializable;
- ausencia de dependencia visual;
- unicidad dentro de su registry o snapshot;
- persistencia entre cargas cuando representa una entidad guardada.

Los IDs de contenido pueden ser legibles y estables. Los IDs de instancias pueden generarse, pero deben persistirse.

5.3. ScriptableObjects como datos de authoring

Se usan assets para:

- productos;
- proveedores;
- displays;
- perfiles de cliente;
- settings de spawning;
- economía;
- día;
- checkout;
- save;
- UI/UX;
- catálogos de presentación;
- paletas;
- audio;
- registry de assets representativos.

Un asset debe convertirse a un modelo de dominio o configuración validada. El dominio no debe leer directamente campos de `ScriptableObject` durante cada operación.

5.4. Validación de catálogos

Cada catálogo debe comprobar:

- IDs únicos;
- referencias no nulas;
- categorías válidas;
- valores no negativos;
- capacidades positivas;
- monedas compatibles;
- prefabs con componentes requeridos;
- ausencia de entradas duplicadas;
- disponibilidad de fallback cuando se permita;

- compatibilidad con save existente.

La validación debe estar disponible como test y, cuando aporte valor, como comando de Editor.

5.5. RuntimeAssetRegistry

El registry representativo resuelve IDs de contenido hacia prefabs o recursos visuales. Su función es sustituir presentación conservando el mismo root funcional y el mismo ID lógico.

No debe:

- crear reglas de negocio;
- decidir cantidades;
- asignar IDs nuevos a entidades persistidas;
- alterar el snapshot;
- inferir categorías por nombre del prefab.

5.6. Versionado de contenido

Cambiar una definición persistida requiere evaluar:

- si el ID se conserva;
- si el save antiguo puede resolverla;
- si cambió capacidad o categoría;
- si debe existir una migración;
- si el contenido retirado necesita fallback;
- si una partida puede cargar sin ese asset.

Eliminar un asset cuyo ID aparece en un save no debe producir una excepción no controlada. La política puede ser migrar, sustituir, marcar como legado o bloquear la carga con diagnóstico, pero debe ser explícita.

6. Input y control contextual

6.1. Contextos

El modelo actual define:

- `None`;
- `UI`;
- `Gameplay`.

La regla es que UI y Gameplay no estén activos simultáneamente de forma que procesen la misma acción. Un panel modal, una confirmación o un menú debe cambiar el contexto o aplicar un gate específico.

6.2. Input Actions

El asset observado contiene mapas `Player` y `UI`. Los adapters del proyecto deben exponer acciones semánticas propias, no propagar indiscriminadamente todo el asset generado.

Acciones relevantes:

- apuntado/clic;
- destino de movimiento;
- órbita;
- zoom;
- selección;
- confirmar placement;
- rotar;
- cancelar;
- abrir/cerrar gestión;
- navegación UI;
- submit/cancel.

El asset contiene acciones heredadas de plantilla que pueden no tener uso real. Deben clasificarse y eliminarse o documentarse para evitar bindings fantasma.

6.3. EventSystem y supresión de input de mundo

Antes de hacer raycast de mundo, el controlador debe comprobar si el puntero está sobre UI. Si la UI consume el clic:

- no se mueve el personaje;
- no se coloca ni selecciona mobiliario;
- no se interactúa con el mundo;
- no se cambia cámara por drag de UI;
- no se genera feedback de fallo de gameplay.

La comprobación debe cubrir ratón y, cuando corresponda, dispositivos apuntadores adicionales.

6.4. Conflicto movimiento/placement

Cuando placement está activo:

- clic primario confirma o selecciona según estado;
- clic secundario o cancel abandona;
- rotación modifica preview;
- el mismo clic no crea un destino de movimiento;
- la cámara conserva las acciones permitidas;
- un panel modal bloquea placement.

`GameplayDestinationInputPolicy` y los drivers deben mantener esta separación.

6.5. Lectura por frame

Los adapters pueden producir un frame inmutable con los valores de input de ese ciclo. Las capas internas consumen intención, no dispositivos concretos.

6.6. Rebinding y accesibilidad

La arquitectura debe permitir:

- rebinding futuro;
- sensibilidad de cámara;
- inversión de ejes;

- velocidad de scroll;
- escalado UI;
- reducción de movimiento;
- toggle/hold cuando sea aplicable.

Los settings deben persistirse fuera del save de partida cuando sean preferencias globales.

7. Movimiento, cámara y navegación

7.1. Movimiento por clic

La cadena recomendada es:

```
Input adapter
→ política de contexto
→ raycast contra superficie válida
→ destino normalizado
→ cálculo o solicitud de ruta
→ agente de presentación
→ avance planar
→ llegada / cancelación / recuperación
```

`PlanarMovementCalculator` contiene cálculo determinista; `ClickToMoveAgent` aplica el resultado a Unity.

7.2. Reglas de destino

Un destino se rechaza cuando:

- el puntero está sobre UI;
- no existe superficie válida;
- el estado bloquea movimiento;
- el destino está fuera de zona navegable;
- placement consume el input;
- el agente está deshabilitado;
- la ruta no puede calcularse.

El fallo debe ser silencioso cuando la intención no era mover y visible cuando el usuario necesita comprender por qué no llega.

7.3. Cámara orbital

La cámara se modela mediante estado y constraints:

- yaw;
- pitch si está permitido;
- distancia;
- objetivo;
- límites;
- sensibilidad;
- zoom.

`OrbitCameraCalculator` debe permanecer libre de Unity cuando sea posible. `OrbitCameraRig` aplica transformaciones y resuelve colisiones o smoothing visual.

7.4. Navegación de agentes

El dominio decide objetivos y estados. NavMesh resuelve movimiento espacial. Un cliente no debe almacenar lógica comercial dentro del `NavMeshAgent`.

Un agente debe soportar:

- ruta aceptada;
- llegada;
- ruta parcial;
- ruta inválida;
- timeout;
- replanificación;
- cancelación;
- liberación de reservas al abandonar;
- salida segura durante cierre.

7.5. Construcción y NavMesh

La colocación puede alterar circulación. La solución debe evitar rebakes costosos por cada preview. Opciones válidas:

- obstacles con carving;
- superficies actualizables por lotes;
- validación lógica previa;
- rebake solo al confirmar cambios relevantes;
- límites de muebles y densidad.

La estrategia elegida debe medirse en la escena representativa.

8. Grid, placement, ocupación y acceso

8.1. Modelo lógico

La cuadrícula usa celdas de `0,5 m`. Los tipos principales incluyen:

- `GridCoordinate`;
- `GridSize`;
- `GridRotation`;
- `GridFootprint`;
- `GridBounds`;
- `GridWorldPosition`.

Las coordenadas lógicas deben ser enteras. La conversión mundo-grid debe estar centralizada y ser consistente entre preview, confirmación, carga y tests.

8.2. Rotación

Las orientaciones permitidas son:

- 0°;
- 90°;
- 180°;

- 270°.

La rotación modifica la huella determinísticamente. No se admiten escalas arbitrarias como sustituto de una huella.

8.3. PlacementOccupancyRegistry

El registry mantiene la relación entre:

- `PlacementInstanceId`;
- definición técnica;
- coordenada;
- rotación;
- celdas ocupadas.

Invariantes:

- una celda no tiene dos ocupantes;
- un ID no se registra dos veces;
- una huella fuera de bounds no se confirma;
- consultar un candidato no muta;
- retirar libera todas y solo sus celdas;
- un fallo deja el registry intacto.

8.4. Secuencia de placement

```
seleccionar definición
→ crear preview
→ proyectar puntero
→ snap al grid
→ aplicar rotación
→ calcular huella
→ validar bounds
→ validar overlap
→ validar zona
→ validar acceso
→ mostrar estado
→ confirmar
→ registrar ocupación
→ crear o mover vista
→ persistir
```

La vista no debe aparecer como colocada antes de que el registro lógico confirme.

8.5. Preview

El preview comunica:

- posición final;
- rotación;
- huella;
- válido/inválido;
- causa principal de fallo;
- coste cuando corresponda;
- impacto de acceso si es relevante.

No debe usar un algoritmo distinto al commit. La misma entrada debe producir el mismo resultado salvo que el estado cambie entre preview y confirmación; en ese caso se revalida.

8.6. Validación de acceso

`GridAccessValidator` y `StorePlacementAccessValidator` protegen:

- celdas de entrada reservadas;
- apertura mínima;
- anchors obligatorios;
- alcanzabilidad ortogonal;
- ausencia de bloqueo total.

Causas tipadas observadas:

- celda bloqueada fuera de bounds;
- entrada reservada bloqueada;
- ausencia de entrada abierta;
- anchura insuficiente;
- anchor bloqueado;
- anchor inalcanzable.

La validación de acceso usa la ocupación candidata sin mutar el registro real.

8.7. StoreShellSpecification

La especificación vigente de tienda usa aproximadamente 10×15 m, equivalentes a 20×30 celdas de $0,5$ m. Las zonas, entradas y anchors deben definirse explícitamente.

8.8. Persistencia de placement

El save debe incluir por objeto dinámico:

- ID de instancia;
- ID de definición;
- coordenada;
- rotación;
- estado relevante;
- relación con inventario o display cuando corresponda.

Los objetos estáticos autorados no deben duplicarse al cargar. El contexto de escena distingue estático inicial de dinámico persistido.

9. Productos y catálogos comerciales

9.1. ProductDefinition

Una definición de producto incluye como mínimo:

- ID estable;
- categoría;
- tags;
- nombre o clave localizable;
- precio recomendado o datos para resolverlo;
- tamaño/capacidad cuando aplique;
- compatibilidad con displays;

- referencia visual mediante catálogo externo.

La definición no contiene cantidad runtime.

9.2. Registry

`ProductDefinitionRegistry` resuelve IDs y rechaza duplicados. Los casos de uso no deben recorrer assets de Unity para encontrar una definición.

9.3. Catálogo representativo

El vertical slice utiliza seis productos iniciales como contenido representativo. La arquitectura debe admitir ampliar el catálogo sin modificar código de reglas.

9.4. Precio

La resolución de precio debe separar:

- precio base/recomendado;
- precio configurado por la tienda;
- límites de diseño;
- quote de venta;
- currency.

El dinero se representa en unidades enteras mínimas, no `float`.

9.5. Compatibilidad visual

Un cambio de carátula, prefab o material no cambia `ProductDefinitionId`. Si un producto requiere retirar su ID, debe existir una migración o política de fallback.

10. Inventario y transferencias

10.1. Modelo

El sistema usa:

- `InventoryContainerId`;
- `InventoryContainerType`;
- `InventoryCapacity`;
- `InventoryStack`;
- `Quantity`;
- `InventoryContainer`;
- `InventoryTransferService`.

Tipos observados:

- `Generic`;
- `Storage`;
- `Display`;
- `Transit`;
- `Unspecified` únicamente como estado no configurado.

10.2. Invariantes

- cantidades no negativas;
- capacidad no excedida;
- producto conocido;
- stock origen suficiente;
- origen y destino distintos;
- una unidad pertenece a una única ubicación lógica;
- la suma global se conserva salvo entrada, venta, devolución o pérdida explícita;
- una reserva activa no cuenta como disponible para otra compra;
- un fallo no altera ningún contenedor.

10.3. Transferencia atómica

La secuencia es:

1. validar cantidad;
2. validar producto;
3. comprobar origen;
4. comprobar capacidad destino;
5. preparar nuevo estado de ambos;
6. aplicar decremento e incremento como una unidad lógica;
7. devolver resultado;
8. actualizar vistas.

La vista no ejecuta transferencias por su cuenta.

10.4. Capacidad

La capacidad debe tener una unidad coherente. Si se usa cantidad de unidades para el slice, no debe mezclarse posteriormente con volumen sin una migración explícita.

10.5. Auditoría

Las operaciones significativas deben poder asociarse a:

- orden;
- delivery;
- restock task;
- reserva;
- checkout;
- cierre;
- save.

No es obligatorio persistir un log infinito, pero sí conservar suficientes IDs para evitar duplicados y diagnosticar inconsistencias.

11. Proveedores, pedidos, entregas y recepción

11.1. SupplierCatalog

El catálogo relaciona proveedor, productos ofrecidos, costes, límites y condiciones. La UI consulta una proyección; no muta el catálogo.

11.2. PurchaseOrder

Estados observados:

```
Draft → Submitted → Delivered → Received
      ↘ Cancelled cuando la transición lo permita
```

Cada transición valida el estado actual. Una orden entregada o recibida no puede procesarse otra vez.

11.3. Creación

`SupplierOrderService` valida:

- solicitud no vacía;
- productos no duplicados;
- productos ofrecidos;
- límites de cajas;
- cantidades válidas;
- ID único;
- coste resoluble.

11.4. Delivery y ShipmentBox

La entrega conserva:

- ID;
- relación con orden;

- proveedor;
- cajas;
- estado;
- contenido por caja;
- recepción parcial o total.

Estados observados:

- `AwaitingReceipt`;
- `PartiallyReceived`;
- `Received`.

11.5. ReceivingService

Antes de recibir:

- la orden debe estar entregada;
- delivery y order deben coincidir;
- proveedor debe coincidir;
- la caja debe existir y no estar recibida;
- la definición de producto debe existir;
- el destino debe tener capacidad;
- la mutación de inventario debe ser aceptable.

El commit actualiza inventario y estado de caja de forma coherente. Si falla la mutación, no se marca la caja como recibida.

11.6. Economía de recepción

El coste se registra con una clave idempotente. No debe cobrarse dos veces una misma recepción. La moneda del ledger debe coincidir.

11.7. Presentación

El mundo puede mostrar cajas, zona de recepción y feedback, pero la caja visual no es la fuente autoritativa. Destruir o ocultar el GameObject no equivale a recibirla.

12. Displays, asignación y reposición

12.1. DisplayDefinition e instancia

La definición contiene:

- ID;
- categorías permitidas;
- capacidad;
- reglas visuales;
- compatibilidad.

La instancia contiene:

- ID de instancia;
- definición;
- contenedor de inventario asociado;
- producto asignado;
- estado runtime.

12.2. Asignación

Una asignación falla cuando:

- falta la definición de producto;
- la categoría no está permitida;
- el producto ya está asignado de forma inválida;
- existe otro producto asignado;
- el display contiene stock incompatible.

No se limpia una asignación mientras quede stock.

12.3. Reposición

`DisplayRestockService` valida:

- cantidad positiva;
- tipo de contenedor origen permitido;
- producto asignado;
- definición conocida;
- stock origen;
- capacidad disponible;
- transferencia válida.

La operación usa el servicio de inventario. No duplica la lógica de cantidades.

12.4. Retorno

El retorno de stock a almacén también es atómico. Solo puede limpiar asignación si el display queda vacío y la regla lo permite.

12.5. Sincronización visual

La vista del display debe derivarse de:

- producto asignado;
- cantidad;
- capacidad;
- catálogo visual.

No debe crear una unidad lógica por cada mesh salvo que el diseño lo requiera. Puede usar niveles visuales, pooling o slots visuales.

13. Clientes, spawning, navegación y shopping

13.1. CustomerProfile

Un perfil define preferencias, paciencia, comportamiento y referencias de presentación. La instancia runtime mantiene identidad y estado.

13.2. Estados del cliente

Estados de navegación observados:

```
WaitingToEnter
→ Entering
→ Browsing
→ Leaving
→ Despawned
```

El shopping añade estados:

```
Searching
→ HoldingReservations
→ ReadyForCheckout
→ CheckedOut
  o Abandoned
```

La coordinación debe impedir combinaciones imposibles, por ejemplo cliente `Despawned` con reserva activa o `CheckedOut` aún en cola.

13.3. Spawning

`CustomerSpawnService` valida:

- cola de spawn;
- límite de población;
- perfil existente;
- ID de instancia no duplicado;
- consistencia de la solicitud.

El reloj de arrivals debe ser controlable en tests.

13.4. Navegación

`CustomerNavigationPlan` contiene objetivos semánticos como entrada, browse y salida. La capa Unity resuelve posiciones y rutas.

La recuperación de ruta debe:

- reintentar con límite;
- elegir fallback;
- cancelar la intención cuando corresponda;
- liberar reservas;
- permitir drenaje durante cierre;
- evitar loops infinitos.

13.5. Búsqueda y disponibilidad

El cliente evalúa productos disponibles en displays. Debe diferenciar:

- stock físico total;
- stock visible;
- stock reservado;
- stock disponible;
- compatibilidad de perfil;
- precio;
- distancia o coste de búsqueda.

13.6. Reserva lógica

Estados observados:

- `Active`;
- `Released`;
- `Consumed`.

Una reserva:

- pertenece a un cliente y carrito;
- referencia producto y display;
- reserva cantidad válida;
- reduce disponibilidad;
- puede liberarse al abandonar;
- se consume en checkout;

- no puede consumirse o liberarse dos veces.

13.7. Carrito y sesión

El carrito no contiene unidades físicas duplicadas; contiene referencias a reservas activas. Las reglas validan propiedad, capacidad y estado.

13.8. Paciencia y abandono

La paciencia se actualiza mediante servicio determinista. El abandono debe producir una causa y una limpieza completa:

- retirar de cola si procede;
- liberar reservas;
- cerrar sesión;
- planificar salida;
- actualizar feedback y métricas.

13.9. Consistencia global

En cualquier punto:

- un customer ID aparece una sola vez;
- un carrito pertenece a una sesión;
- una reserva activa pertenece a un carrito válido;
- una entrada de cola referencia sesión y carrito válidos;
- una transacción completada no conserva reservas activas.

14. Cola, estación y checkout

14.1. Cola FIFO

Estados de entrada:

- `Waiting;`

- `Called;`
- `Processing;`
- `Completed;`
- `Cancelled.`

La cola debe mantener FIFO salvo regla explícita futura. Se rechazan:

- entrada duplicada;
- cliente ya en cola;
- carrito ya en cola;
- cola llena;
- front ocupado;
- estado inválido;
- cola sellada durante cierre.

14.2. Estación

Estados:

- `Closed;`
- `Available;`
- `Busy.`

La estación busy debe conocer la entrada que procesa. No puede procesar una entrada distinta de la front llamada.

14.3. Preflight de checkout

Antes del commit se valida:

- ID de transacción no usado;
- carrito no cobrado;
- entrada de cola en processing;
- estación en processing y con la misma entrada;
- sesión lista;
- carrito no vacío;
- propiedad;

- cada reserva existe y está activa;
- display existe y mantiene producto;
- stock suficiente;
- quote de precio disponible;
- ledger compatible;
- IDs de postings no duplicados.

14.4. Commit

El commit lógico debe producir exactamente una vez:

1. consumo de stock físico;
2. consumo de reservas;
3. estado del carrito;
4. estado de sesión;
5. avance de cola;
6. liberación de estación;
7. registro de transacción;
8. ingreso en ledger;
9. feedback y métricas.

Si una etapa no puede completarse, no debe quedar un checkout parcialmente confirmado. La implementación puede usar snapshots previos, operaciones preparadas o una transacción de aplicación, pero el comportamiento observable es atómico.

14.5. Cancelación

`CheckoutCancellationService` resuelve salida segura cuando:

- el cliente abandona;
- se cierra la tienda;
- la estación no puede procesar;
- existe un fallo recuperable.

Debe comprobar ownership, estado de reserva y cola antes de liberar.

14.6. Idempotencia

Repetir una solicitud con el mismo transaction/posting ID no genera una segunda venta. La carga de un save posterior a una venta no debe permitir reejecutar el commit por conservar un trigger visual.

15. Ciclo diario, cierre y economía

15.1. Estados de jornada

```
BeforeOpen → Open → Closing → Closed
```

Transiciones inválidas devuelven fallo tipado. El tiempo transcurrido no puede ser negativo.

15.2. BeforeOpen

Permite preparación, pedidos, recepción, reposición, precios, construcción y guardado según las reglas del GDD.

15.3. Open

Habilita spawning, shopping, cola y ventas. Determinadas operaciones de gestión pueden estar limitadas por UX o balance, pero la regla debe estar en Application.

15.4. Closing

Durante cierre:

- se sella la cola o se impiden nuevas entradas;
- se detiene spawning;
- se drenan clientes;
- se completan o cancelan checkouts según política;
- se liberan reservas restantes;
- se cierra estación;
- se construye snapshot de cierre;

- se transita a `Closed` solo si las condiciones se cumplen.

`StoreClosureCoordinator` orquesta estos pasos.

15.5. Closed

Permite resultados, autosave y avance. No deben quedar:

- clientes activos;
- reservas activas;
- cola pendiente;
- estación busy;
- sesiones de shopping abiertas.

15.6. Money

`Money` usa cantidad entera en centavos y código de moneda de tres caracteres. No se realizan operaciones económicas con `float`.

15.7. Ledger

Tipos observados:

- `CheckoutRevenue`;
- `SupplierReceivingCost`.

El ledger protege:

- ID de entrada único;
- clave de posting única;
- moneda consistente;
- trazabilidad de origen.

Los tipos futuros deben añadirse de forma explícita, no usar strings libres.

15.8. Resultados diarios

`DailyResultsService` compara actividad de jornada con transacciones y movimientos. Debe producir un resultado reproducible y detectar discrepancias de conteo.

15.9. Fiscalidad

Las reglas semanales e impuestos descritas en GDD deben implementarse como servicios de aplicación y entradas de ledger. No se introducen cálculos fiscales dentro de UI.

16. Persistencia, slots, backup y recuperación

16.1. Dos niveles observados

La base contiene:

1. sesión/save skeleton inicial (`GameSession` , `GameSessionSnapshot` , `JsonSaveGameRepository`);
2. persistencia integrada del vertical slice (`IntegratedGameStateSnapshot` , records, codec, repository y service).

La arquitectura debe converger hacia una única ruta autoritativa. El skeleton puede mantenerse como compatibilidad o fachada mientras se migra, pero no deben existir dos saves divergentes para el mismo slot.

16.2. Snapshot integrado

El snapshot incluye, entre otros:

- schema version;
- session ID;
- slot;
- timestamps UTC;
- día actual;
- efectivo y moneda;
- inventarios;
- órdenes;
- displays;

- clientes;
- sesiones de shopping;
- reservas;
- entradas de cola;
- estación;
- transacciones;
- ciclo diario;
- ledger.

Los objetos de escena y placement deben formar parte del snapshot definitivo o de un subestado compatible antes de cerrar el gate.

16.3. Validación cruzada

Al construir o cargar se valida:

- schema soportado;
- UTC correcto;
- `UpdatedUtc >= CreatedUtc`;
- día positivo;
- moneda válida;
- IDs únicos;
- referencias entre displays e inventarios;
- clientes existentes;
- carritos únicos;
- reservas coherentes;
- cola y estación coherentes;
- transacciones y ledger válidos;
- ausencia de referencias huérfanas.

16.4. Escritura atómica

El patrón obligatorio es:

```
serializar en memoria
→ validar payload
→ escribir archivo temporal
→ flush/cierre
→ preservar backup anterior
→ reemplazo atómico o rename seguro
→ verificar existencia
→ actualizar descriptor de slot
```

Un fallo no debe destruir el último save válido.

16.5. Estados de repository

Se contemplan:

- `Success`;
- `SlotEmpty`;
- `RecoveredFromBackup`;
- `ValidationFailure`;
- `UnsupportedSchema`;
- `CorruptPrimaryNoBackup`;
- `StorageFailure`.

La UI debe distinguirlos. “Slot vacío” no es error; “schema no soportado” no debe presentarse como corrupción genérica.

16.6. Backup y recovery

Si el primario falla y el backup es válido:

- se carga el backup;
- se marca el slot como recuperado;
- se informa al usuario;
- no se sobrescribe inmediatamente sin una acción segura;
- se conserva diagnóstico del primario.

16.7. Migraciones

Cada schema nuevo debe definir:

- versión origen;
- versión destino;
- transformación;
- valores por defecto;
- validación posterior;
- tests con fixtures antiguos;
- política para versiones demasiado antiguas.

No se cambia `CurrentSchemaVersion` sin implementar y probar la ruta necesaria.

16.8. Autosave

El autosave ocurre en un punto seguro, preferentemente tras cierre y resultados. Debe impedir:

- dos saves concurrentes;
- guardar durante mutación crítica;
- marcar éxito antes de completar repository;
- repetir el mismo autosave de jornada sin razón.

16.9. Rutas y archivos

Las rutas se construyen con APIs seguras. El nombre de slot no acepta traversal. Los datos de usuario no se almacenan en Assets. Los logs no contienen el JSON completo salvo modo técnico controlado.

17. UI, tutorial, accesibilidad y proyecciones

17.1. Separación

La UI consume modelos de lectura y emite comandos. No accede directamente a listas mutables de dominio ni a repositorios concretos.

17.2. Capas UI

Estados observados:

- HUD;
- panel de gestión;
- submenú;
- tooltip;
- confirmación;
- pausa.

Una capa superior define qué input queda activo. Las confirmaciones bloquean acciones subyacentes.

17.3. Paneles de gestión

IDs observados:

- Inventory;
- Suppliers;
- Displays;
- Customers;
- Shopping;
- Checkout;
- DayCycle;
- Economy;
- Help;
- Accessibility.

La UI puede agrupar o renombrar visualmente estos paneles, pero la navegación debe conservar estado explícito.

17.4. StoreUiProjectionService

La proyección traduce estado complejo a datos de presentación. Debe:

- evitar exponer referencias mutables;
- resolver textos en capa adecuada;

- no ejecutar side effects;
- ser reproducible;
- permitir pruebas EditMode.

17.5. Tutorial

Pasos observados:

- bienvenida;
- movimiento y cámara;
- gestión;
- inventario;
- proveedores;
- displays;
- clientes y shopping;
- cola y checkout;
- ciclo diario;
- resultados;
- autosave;
- completado.

El tutorial persiste progreso global o por partida según decisión explícita. Saltarlo no debe bloquear capacidades.

17.6. Accesibilidad

La base contempla settings. El TDD exige que toda nueva UI considere:

- navegación por teclado;
- foco visible;
- contraste;
- escala;
- no depender solo del color;
- textos localizables;
- reducción de movimiento cuando aplique;

- feedback redundante visual/sonoro;
- tiempos y confirmaciones razonables.

17.7. MainMenu y slots

Los descriptors de slot distinguen:

- Empty;
- Ready;
- Recovered;
- Corrupt;
- UnsupportedSchema;
- StorageFailure.

La acción de sobrescribir requiere confirmación. La UI no debe ocultar una recuperación de backup.

18. StoreInitial y assets representativos

18.1. Problema de la solución procedural

`Phase1StoreBlockoutBuilder` y `RepresentativeStoreVisualBuilder` surgieron para derivar una presentación desde placeholders técnicos. La baseline v0.6 identifica problemas:

- bounds no fiables;
- escalas heredadas;
- ejes inconsistentes de FBX;
- jerarquías accidentales;
- puertas y muros difíciles de alinear;
- duplicación visual;
- divergencia entre shell lógico y representación.

Estos builders son TRANSITORIOS. No deben convertirse en arquitectura final de la tienda.

18.2. Jerarquía objetivo

```
StoreInitial
├── Systems
├── Cameras
├── UI
├── StoreEnvironment
├── DynamicFurniture
├── DynamicProducts
├── Customers
└── Debug
```

La jerarquía puede variar de nombre, pero debe mantener fronteras equivalentes.

18.3. StoreInitialEnvironment.prefab

Debe contener:

- shell visual;
- suelos y paredes;
- puertas y piezas animables;
- iluminación local;
- colliders técnicos;
- mobiliario estático aprobado;
- anchors;
- roots de decoración;
- marcadores de zonas cuando sean authoring.

No debe contener:

- `ApplicationRoot`;
- HUD global;
- `EventSystem` duplicado;
- cámara persistente;
- save repository;
- managers de gameplay;
- lógica económica;

- catálogos globales duplicados.

18.4. StoreInitialSceneContext

Es un componente objetivo con referencias serializadas a:

- placement surface;
- player spawn;
- entrance;
- checkout;
- receiving;
- backroom;
- door controller;
- root de mobiliario inicial;
- root de mobiliario dinámico;
- root de productos;
- customer spawns;
- lighting root;
- anchors obligatorios;
- contexto de cámara si procede.

La resolución debe fallar temprano si falta una referencia obligatoria.

18.5. Secuencia runtime objetivo

```
cargar escena
→ resolver StoreInitialSceneContext
→ validar referencias e IDs
→ registrar superficie y anchors
→ registrar mobiliario inicial
→ configurar puerta
→ resolver catálogos
→ restaurar snapshot
→ crear solo entidades dinámicas
→ sincronizar visuales
→ iniciar día, clientes y UI
```

No se construye el shell en runtime.

18.6. Mobiliario inicial y dinámico

- El mobiliario inicial autorado tiene IDs estables.
- El mobiliario dinámico procede del save o de acciones del jugador.
- Al cargar no se duplica un ID existente en escena.
- Un objeto inicial retirado o movido necesita una política persistente explícita.
- Los roots permiten limpiar dinámicos sin destruir el entorno.

18.7. Puerta

La puerta visible puede usar `AutomaticSlidingDoorController` y partes representativas, pero la autorización de acceso no depende del nombre de meshes. El contexto registra controller, threshold y anchors.

18.8. Factories representativas

Las factories sustituyen la presentación conservando:

- ID;
- root funcional;
- colliders requeridos;
- puntos de interacción;
- anchors;
- relación con el modelo.

No pueden cambiar la huella lógica basándose en el prefab salvo que la definición técnica se actualice y valide.

18.9. Migración segura

Fases:

1. mantener `Store` y builders como fallback;
2. completar contexto de `StoreInitial`;
3. validar catálogos y prefabs;
4. registrar objetos iniciales;

5. integrar snapshot;
6. ejecutar suites;
7. completar Golden Path;
8. comparar estado antes/después;
9. incluir `StoreInitial` en build de prueba;
10. validar Windows x64 y `Player.log`;
11. cambiar ruta principal;
12. retirar fallbacks en un cambio posterior independiente.

No se borra código procedural en la misma iteración que activa la nueva escena, salvo que exista rollback probado.

18.10. Criterios de ausencia de duplicados

Después de cargar:

- un solo `ApplicationRoot`;
- un solo EventSystem activo;
- una sola cámara principal efectiva;
- un solo runtime root de tienda;
- un ID por fixture;
- un contenedor por ID;
- un checkout station lógico;
- una puerta controladora por acceso;
- ningún shell duplicado;
- ningún cliente duplicado por restore + spawn.

19. Herramientas de Editor y authoring

19.1. Instaladores idempotentes

Los instaladores históricos de Sprints 3-5 y las herramientas de organización demuestran un patrón válido para cambios de escena repetibles. Una herramienta idempotente:

- encuentra o crea el objeto esperado;
- no duplica componentes;
- conserva referencias válidas;
- corrige configuración obsoleta;
- puede ejecutarse dos veces con el mismo resultado;
- marca assets modificados;
- guarda solo cuando corresponde;
- produce un resumen.

19.2. Migraciones de organización

Las herramientas de `Editor/ProjectOrganization` deben operar sobre rutas conocidas, validar metadatos y evitar mover assets por heurísticas ambiguas.

19.3. Reparación de fixtures

Una reparación de prefab o catálogo debe:

- identificar versión/estado anterior;
- realizar cambios mínimos;
- validar componentes requeridos;
- preservar GUID cuando sea posible;
- evitar romper referencias de escenas;
- ejecutar tests de authoring.

19.4. Validadores

Se recomiendan validadores para:

- build scene list;
- assemblies;
- catálogos;
- StoreInitial context;
- IDs duplicados;
- prefabs representativos;

- colliders y layers;
- referencias de input;
- settings de save;
- localización;
- materiales URP;
- fixtures iniciales.

19.5. Límites

Las herramientas de Editor no deben ser requisito para que una build cargue una partida. Su función es producir y validar assets, no ejecutar lógica runtime oculta.

20. Estrategia de pruebas

20.1. Pirámide

Domain/Application — EditMode

Prueban:

- value objects;
- estados;
- transiciones;
- invariantes;
- servicios;
- fallos tipados;
- idempotencia;
- snapshots;
- proyecciones;
- migraciones puras.

Infrastructure — EditMode

Prueban:

- codecs;
- repositorios temporales;
- catálogos;
- authoring assets;
- atomicidad de archivo;
- recovery;
- configuración.

Presentation/Integration — PlayMode

Prueban:

- componentes;
- escenas;
- cámara;
- input;
- placement visual;
- contextos;
- UI;
- integración runtime;
- ausencia de duplicados.

Build/Golden Path — manual y automatizable

Prueban el producto fuera del Editor.

20.2. Volumen observado

La instantánea contiene 143 archivos de test y aproximadamente 1.270 atributos de prueba. La cantidad es una evidencia favorable, pero el gate depende de cobertura de riesgos, no del número bruto.

20.3. Convenciones

Cada test debe:

- ser independiente;

- usar Arrange/Act/Assert legible;
- nombrar comportamiento y condición;
- no depender de orden;
- limpiar archivos temporales;
- usar reloj/IDs deterministas;
- comprobar ausencia de mutación en fallos;
- evitar esperas reales;
- reportar la causa exacta.

20.4. Cobertura obligatoria por sistema

Placement

- bounds;
- overlap;
- rotación;
- retirada;
- preview/commit;
- acceso;
- persistencia.

Inventario

- cantidad;
- capacidad;
- transferencia;
- conservación global;
- rollback lógico.

Pedidos/recepción

- catálogo;
- transición;
- caja duplicada;
- recepción parcial;

- coste idempotente.

Shopping/checkout

- reservas;
- ownership;
- abandono;
- FIFO;
- station;
- preflight;
- commit;
- cancelación;
- doble venta.

Día/economía

- transiciones;
- cierre;
- drain;
- ledger;
- resultados;
- autosave.

Persistencia

- round trip;
- corrupción;
- backup;
- schema;
- referencias cruzadas;
- equivalencia de estado.

StoreInitial

- context;
- objetos iniciales;

- dinámicos;
- puerta;
- anchors;
- UI suppression;
- save/load;
- build list.

20.5. Golden Path

Debe ejecutarse como mínimo:

```
iniciar partida
→ preparar tienda
→ pedir checkout/display/producto
→ recibir
→ colocar
→ asignar y reponer
→ abrir
→ servir cliente
→ cobrar
→ cerrar
→ revisar resultados
→ autosave
→ salir
→ cargar
→ verificar equivalencia
```

20.6. Datos de prueba

Los fixtures deben usar IDs legibles y cantidades pequeñas. No deben depender de assets de arte salvo pruebas de integración representativa.

20.7. Tests frágiles

Debe evitarse:

- assertions de jerarquía visual innecesariamente exacta;
- tiempos de frames rígidos;
- búsqueda por nombres cuando existe ID/context;

- snapshots textuales gigantes sin intención;
- dependencia de resolución;
- uso de rutas locales del desarrollador.

21. Logging, diagnóstico y manejo de errores

21.1. Categorías

Los logs deben usar prefijos o categorías consistentes:

- SceneFlow;
- Input;
- Placement;
- Access;
- Inventory;
- Supply;
- Receiving;
- Customers;
- Shopping;
- Checkout;
- DayCycle;
- Economy;
- Persistence;
- UIUX;
- StoreInitial;
- Build.

21.2. Niveles

- información: transición significativa;
- warning: fallback, recuperación o dato no crítico;
- error: operación no completada o configuración inválida;

- exception: contrato roto o fallo inesperado.

No se registra cada tick, movimiento o frame en producción.

21.3. Contexto mínimo

Un error relevante incluye:

- operación;
- ID principal;
- estado;
- causa tipada;
- escena;
- slot cuando proceda;
- schema cuando proceda.

No debe incluir datos personales ni payload completo sin necesidad.

21.4. Errores recuperables

Se devuelven como resultados y feedback. Ejemplos:

- slot vacío;
- placement inválido;
- stock insuficiente;
- cola llena;
- destino no alcanzable;
- save recuperado desde backup.

21.5. Errores no recuperables

Deben bloquear o degradar de forma segura:

- IDs duplicados de contenido;
- StoreInitial context incompleto;
- schema no soportado;

- snapshot inconsistente;
- escena obligatoria ausente;
- catálogo esencial no cargado;
- transacción parcial imposible de restaurar.

21.6. TechnicalScenarioRunner

Los runners técnicos son útiles para reproducir escenarios, pero no sustituyen pruebas. Deben estar desactivados en builds no técnicas o activarse mediante configuración explícita.

22. Rendimiento y memoria

22.1. Presupuestos iniciales

Métrica	Objetivo del vertical slice
Resolución de referencia	1920 × 1080
Frame rate	60 FPS
Clientes simultáneos	8
Tiempo de carga del slice	< 5 s en equipo de desarrollo
Guardado/cierre	< 2 s en equipo de desarrollo
Allocations estables	Próximas a cero por frame en simulación estable
Errores/excepciones recurrentes	0

22.2. Reglas

- no usar LINQ en loops por frame sin medir;
- no crear strings de log continuamente;
- distribuir IA por ticks o eventos;
- no recorrer todos los GameObjects para cada operación;
- cachear referencias resueltas;

- usar pooling para VFX, clientes o elementos recurrentes cuando el profiler lo justifique;
- evitar instanciar productos visuales unitarios en exceso;
- batch de sincronización tras cargas;
- no serializar el save en cada cambio pequeño;
- no rebakear navegación por preview.

22.3. Perfilado

Antes de cerrar Sprint 17 se debe medir:

- CPU main thread;
- scripts;
- rendering;
- batches y setpass;
- GC allocations;
- memoria de texturas/meshes;
- tiempo de save/load;
- spawning;
- cierre de día;
- sincronización visual;
- carga de StoreInitial.

Las capturas deben identificar equipo, build, escena, población y duración.

22.4. Escalabilidad

La arquitectura debe admitir más clientes y contenido, pero no se optimiza prematuramente para cientos de agentes. Cada aumento se valida con un presupuesto nuevo.

23. Build, versión y distribución interna

23.1. Plataforma

Target inicial: Windows x64. No se promete soporte para otras plataformas hasta una decisión formal.

23.2. Preflight de build

Antes de generar:

- proyecto compila sin errores;
- paquetes bloqueados;
- escenas correctas y habilitadas;
- `StoreInitial` solo si supera gate;
- catálogos validados;
- tests obligatorios verdes;
- settings de input asignados;
- no hay assets Editor en runtime;
- versión actualizada;
- ruta de salida limpia;
- no hay secretos o rutas locales.

23.3. Build de desarrollo

Puede incluir:

- Development Build;
- Script Debugging cuando se necesite;
- logs ampliados;
- escenario runners controlados;
- overlay técnico.

Debe distinguirse de una build candidata a distribución.

23.4. Validación externa al Editor

Se ejecuta:

1. arranque desde ejecutable;
2. MainMenu;
3. nueva partida;
4. Golden Path;
5. autosave;
6. cierre del proceso;
7. carga del slot;
8. verificación de estado;
9. revisión de `Player.log`;
10. repetición con slot recuperado si procede.

23.5. Scene list

La escena principal que no figure en build no está cerrada. Los tests deben comprobar GUID, ruta, enabled y orden cuando el orden sea contractual.

23.6. Versionado

La versión de aplicación y la versión de schema son conceptos distintos. Cambiar una no obliga a cambiar la otra, pero ambas deben registrarse en diagnósticos.

23.7. Evidencias

El cierre conserva:

- hash/commit de código;
- versión Unity;
- manifest de paquetes;
- resultado de tests;
- build path;
- fecha;

- equipo de prueba;
- `Player.log`;
- incidencias conocidas;
- resultado del Golden Path.

24. Seguridad de datos e integridad

Aunque es un juego local, deben protegerse:

- rutas de save;
- reemplazo atómico;
- backups;
- validación de JSON;
- límites de tamaño razonables;
- rechazo de schema desconocido;
- IDs y referencias;
- ausencia de traversal;
- tratamiento de archivos bloqueados;
- fallos por disco lleno o permisos.

El loader no debe ejecutar tipos ni código descritos por el save. Se serializan DTOs conocidos.

Las herramientas de importación futura deben tratar datos externos como no confiables y validarlos antes de convertirlos a modelos.

25. Arquitectura de sistemas futuros

25.1. Regla general

Los sistemas futuros se diseñan como contextos que reutilizan IDs, inventario, economía, tiempo, tareas y persistencia. No se implementan hasta superar los gates de producto.

25.2. Empleados — DIFERIDO

Necesitarán:

- EmployeeId;
- perfil y contrato;
- salario;
- disponibilidad;
- skills;
- fatiga;
- cola de tareas;
- reservas de recursos;
- prioridades;
- interrupción segura;
- persistencia.

No deben controlar inventario directamente; ejecutan casos de uso existentes.

25.3. Investigación — DIFERIDO

Necesitará nodos, requisitos, costes, duración y unlocks. Los desbloques modifican catálogos o políticas, no `if` dispersos en vistas.

25.4. Puestos informáticos — DIFERIDO

Deben reutilizar:

- placement;
- componentes/inventario;
- clientes;
- reservas;
- tiempo;
- economía;
- mantenimiento;
- electricidad si se aprueba.

25.5. Comercio online — MEDIO PLAZO

Debe compartir catálogo, stock, precio, pedidos y logística. Un pedido online reserva stock de forma explícita; no mantiene un inventario mágico paralelo.

25.6. Publishing y desarrollo — VISIÓN TARDÍA

Requieren contextos separados y no deben contaminar Domain actual con tipos prematuros. Sus contratos podrán usar Money, tiempo, IDs y ledger comunes.

25.7. Plataforma e infraestructura — VISIÓN TARDÍA

La simulación será abstracta y local salvo decisión contraria. No se diseñan servicios online reales como requisito del juego ficticio.

26. Deuda técnica y migraciones conocidas

26.1. StoreInitial fuera de build

Estado: deuda/gate esperado.

Salida: completar contexto, pruebas, Golden Path y build antes de habilitarla.

26.2. Builders procedurales

Estado: fallback transitorio.

Riesgo: geometría deducida, duplicados y divergencia.

Salida: autoría explícita y retirada posterior en cambio independiente.

26.3. Sprint16Phase1RuntimeRoot global

Estado: root transitorio auto-instalado antes de escena.

Riesgo: ciclo de vida global para una composición específica, duplicados y acoplamiento.

Salida: mover composición a contexto de escena o root de tienda validado.

26.4. Inyección por escaneo de escena

Estado: solución de fundación.

Riesgo: coste, dependencia implícita y difícil diagnóstico.

Salida: composición explícita por contexto manteniendo interfaces de consumidor durante la migración.

26.5. Dependencia Infrastructure.InputSystem → Presentation

Estado: tolerada.

Riesgo: inversión de frontera.

Salida: extraer contratos/frames o conectar desde composition root.

26.6. Save skeleton e integrated save

Estado: coexistencia histórica.

Riesgo: dos fuentes de verdad.

Salida: definir fachada única y migrar slots sin pérdida.

26.7. Acciones de plantilla no usadas

Estado: asset Input Actions contiene acciones genéricas.

Riesgo: bindings ambiguos y mantenimiento.

Salida: inventariar uso, eliminar o documentar.

26.8. Duplicación de relojes UTC

Existen adaptadores `SystemUtcClock` en más de un namespace. Debe evaluarse consolidación sin romper contratos.

26.9. Store y StoreInitial similares

Las escenas contienen estructura técnica parecida. No deben evolucionar en paralelo durante mucho tiempo. Tras el gate se elige una ruta principal y se mantiene una política clara para la histórica.

26.10. TechnicalScenarioRunners

Deben permanecer controlados y fuera de builds finales, evitando lógica duplicada.

26.11. Priorización

Prioridad	Deuda	Condición de cierre
P0	Duplicados o pérdida de save	Antes de activar StoreInitial
P0	StoreInitial sin contexto completo	Gate Sprint 16
P1	Build list y regresión	Gate Sprint 17
P1	Root transitorio/procedural	Tras migración segura
P2	Dependencias de assembly	Antes de ampliar Input/UI
P2	Acciones y adaptadores duplicados	Limpieza post-slice
P3	Organización y nombres históricos	Mantenimiento programado

27. ADR, control de cambios y revisión técnica

27.1. Cuándo crear ADR

- nueva dependencia de paquete;
- nueva capa o assembly;
- cambio de persistencia;
- cambio de schema;
- modificación de autoridad de escena;
- sustitución de navigation strategy;
- introducción de ECS/DOTS, Addressables u otra tecnología estructural;
- cambio de input/UI framework;
- retirada de fallback importante;
- cambio del modelo de inventario o dinero;
- introducción de servicios online reales.

27.2. Contenido mínimo

- contexto;

- problema;
- alternativas;
- decisión;
- consecuencias;
- riesgos;
- plan de migración;
- rollback;
- pruebas;
- documentos afectados.

27.3. Revisión de código

Aunque el flujo sea individual, cada cambio estructural debe revisarse mediante checklist:

- dirección de dependencias;
- invariantes;
- mutaciones atómicas;
- resultados tipados;
- tests;
- logs;
- persistencia;
- rendimiento;
- authoring;
- documentación.

27.4. Definition of Done técnica de una feature

1. Requisitos y alcance identificados.
2. Diseño coherente con capas.
3. Sin dependencia prohibida.
4. Validación antes de mutación.
5. Fallos tipados.
6. Tests positivos, negativos e invariantes.

7. Integración PlayMode cuando corresponde.
8. Persistencia actualizada o declarada no afectada.
9. UI/input revisados.
10. Logs accionables.
11. Rendimiento medido cuando existe riesgo.
12. Documentación y trazabilidad actualizadas.
13. Build ejecutada si el gate lo exige.
14. Sin S0/S1 abiertos.

28. Criterio de aceptación del TDD

Este TDD se considera operativo cuando:

- las capas y assemblies descritos coinciden con la solución o su deuda está registrada;
- los sistemas del vertical slice tienen contratos técnicos suficientes;
- `StoreInitial` tiene una ruta de migración segura;
- persistencia, input, escenas y build tienen reglas explícitas;
- los criterios del Vertical Slice pueden mapearse a componentes y pruebas;
- no existen contradicciones no resueltas con Enfoque, GDD o Vertical Slice Specification;
- los sistemas futuros están limitados y no aparecen como backlog inmediato;
- una persona puede implementar o revisar cambios sin depender de conocimiento tácito.

28.1. Gate técnico de Sprint 16

- contexto autorado completo;
- arquitectura y referencias explícitas;
- assets representativos integrados;
- ausencia de duplicados;
- compatibilidad con sistemas funcionales;
- pruebas de context, roots, anchors y puerta;
- deuda visual clasificada;
- fallback conservado.

28.2. Gate técnico de Sprint 17

- suites verdes;
- Golden Path completo;
- save/load equivalente;
- rendimiento dentro de presupuesto o desviación aprobada;
- StoreInitial en lista de build cuando corresponda;
- Windows x64 generada;
- `Player.log` revisado;
- S0/S1 en cero;
- evidencias y trazabilidad completas.

28.3. Condición para ampliar arquitectura

No se inicia un contexto diferido hasta que:

- el gate anterior está cerrado;
- existe alcance aprobado;
- se define contrato con sistemas existentes;
- se conoce impacto de save;
- existen criterios de aceptación;
- se actualizan GDD, TDD, modelo de datos y QA.

29. Comunicación entre sistemas, eventos y proyecciones

29.1. Regla de comunicación

La comunicación se divide en cuatro formas y cada una tiene un propósito distinto:

1. **Llamada directa a caso de uso** para una intención síncrona del usuario o de un coordinador.
2. **Resultado tipado** para comunicar éxito, fallo y datos de la operación.
3. **Evento de dominio o aplicación** para informar de un hecho ya confirmado.
4. **Proyección de lectura** para que UI y presentación consulten un estado preparado.

No debe usarse un bus global como sustituto de dependencias explícitas. Los eventos no deben ocultar la secuencia de una transacción crítica.

29.2. Comandos

Un comando representa intención y puede fallar. Ejemplos:

- colocar mueble;
- crear pedido;
- recibir caja;
- asignar producto;
- reponer display;
- abrir tienda;
- reservar unidad;
- entrar en cola;
- confirmar checkout;
- cerrar día;
- guardar slot.

El comando debe contener datos suficientes, pero no referencias a vistas. Los casos de uso resuelven entidades por ID y devuelven resultados.

29.3. Eventos

Un evento describe un hecho ocurrido, no una petición. Ejemplos válidos:

- `ObjectPlaced;`
- `OrderSubmitted;`
- `DeliveryReceived;`
- `ProductReserved;`
- `CheckoutCompleted;`
- `StoreClosingStarted;`
- `DayClosed;`
- `SaveCompleted.`

Reglas:

- se emiten después del commit;
- son inmutables;
- incluyen IDs y datos mínimos;
- no permiten que un listener revierta silenciosamente la operación;
- un fallo de presentación no invalida el hecho de dominio ya confirmado;
- los listeners se suscriben y desuscriben de forma explícita;
- los eventos persistentes no dependen de referencias Unity.

29.4. Eventos versus coordinación transaccional

Checkout, recepción y cierre requieren una coordinación explícita. No deben implementarse como una cadena de listeners donde cada sistema muta por separado. El coordinador ejecuta el preflight, prepara los cambios, confirma la transacción y después publica eventos para UI, audio, VFX o telemetría.

29.5. Proyecciones

Las proyecciones de lectura deben:

- contener copias o estructuras inmutables;
- agrupar datos de varios agregados sin conceder acceso mutable;
- poder regenerarse;
- incluir estado de disponibilidad y razones de bloqueo;
- evitar que UI replique reglas;
- soportar localización mediante claves o modelos de texto apropiados.

29.6. Frecuencia de actualización

No toda UI necesita refresco por frame. Se recomienda:

- eventos para cambios discretos;
- polling de baja frecuencia para reloj o métricas continuas;
- invalidación de proyección tras mutación;
- batch de actualizaciones al restaurar un save;

- evitar reconstrucciones completas por cada unidad transferida cuando puede agruparse.

30. Tiempo, concurrencia y orden de ejecución

30.1. Hilo principal

Las APIs de Unity y la mayoría de componentes de escena se operan en el hilo principal. El hecho de usar tareas o I/O asíncrono no permite tocar GameObjects fuera de él.

30.2. Operaciones asíncronas

Son candidatas:

- carga de escena;
- lectura/escritura de archivo cuando se justifique;
- carga futura de assets;
- operaciones de plataforma.

Deben incluir:

- cancelación o invalidación al cambiar escena;
- captura explícita de errores;
- retorno al hilo principal para aplicar presentación;
- gate contra operaciones concurrentes;
- indicador UI cuando el usuario espera.

30.3. Reentrada

Los servicios no deben asumir que una UI no enviará dos veces una acción. Deben proteger:

- doble clic en cargar;
- doble confirmación de pedido;
- doble recepción;
- doble checkout;

- doble cierre;
- doble guardado;
- transición de escena concurrente.

La UI puede deshabilitar botones, pero la capa de aplicación mantiene la defensa autoritativa.

30.4. Orden de inicialización

Se emplean execution order o roots explícitos únicamente cuando es necesario. La regla objetivo es que una dependencia se entregue de forma explícita, no confiar en que `Awake` de otro objeto haya ocurrido por casualidad.

Secuencia recomendada de escena:

1. validar contexto;
2. resolver servicios globales;
3. construir servicios de escena;
4. cargar o crear estado;
5. registrar entidades autoradas;
6. restaurar dinámicos;
7. vincular vistas;
8. activar input;
9. iniciar simulación;
10. emitir estado listo.

Durante los pasos 1–8 el usuario no debe poder mutar gameplay.

30.5. Pausa

La pausa no debe depender únicamente de `Time.timeScale = 0`. Debe definir:

- simulación detenida;
- input de gameplay bloqueado;
- UI activa;
- audio atenuado según diseño;
- coroutines y timers que usan tiempo escalado/no escalado;
- operaciones de archivo permitidas;

- estado de navegación de agentes.

30.6. Reloj de juego y reloj UTC

El reloj de juego gobierna jornada y simulación. UTC se utiliza para metadatos de save, ordenación de slots y diagnóstico. No deben mezclarse. Una jornada no avanza por el reloj real del sistema salvo diseño explícito.

31. Localización, audio, VFX y feedback técnico

31.1. Localización

La arquitectura debe soportar ES/EN sin almacenar texto final en Domain. Las causas de fallo se representan mediante enums o claves; Presentation resuelve el texto.

Reglas:

- no concatenar frases localizables con orden fijo;
- usar placeholders nombrados o posicionales controlados;
- formatear dinero, números y fechas según locale;
- probar expansión de texto;
- evitar imágenes con texto incrustado;
- persistir idioma como preferencia global;
- disponer de fallback cuando falta una entrada.

31.2. Audio

`Phase1AudioCatalogAsset` y `Phase1AudioRouter` forman la base representativa. El audio debe reaccionar a eventos confirmados, no mutar gameplay.

Canales observados:

- Music;
- Ambience;
- Ui;
- Effects.

El router debe:

- resolver clips por ID o evento;
- respetar volúmenes;
- evitar crear AudioSources sin límite;
- usar pooling o fuentes compartidas;
- no reproducir varias veces por un mismo evento;
- tolerar clip ausente con warning controlado.

31.3. VFX

`Phase1VfxPool` indica una estrategia de reutilización. Los VFX:

- son efímeros;
- no poseen estado autoritativo;
- se activan tras eventos;
- deben devolver instancia al pool;
- no deben bloquear una operación si faltan;
- respetan presupuesto de partículas y overdraw.

31.4. Feedback

Los tipos representativos incluyen placement válido/inválido, selección, asignación, falta de stock, reserva, reposición, recepción, satisfacción, frustración, cola, checkout, ingreso, gasto, cierre y autosave.

Cada feedback debe identificar:

- evento origen;
- prioridad;
- canal visual/sonoro;
- duración;
- posibilidad de agregación;
- comportamiento cuando varios ocurren simultáneamente.

31.5. Accesibilidad del feedback

Un estado importante no puede depender exclusivamente de sonido, color o animación. Los fallos deben disponer de texto o iconografía legible cuando sea necesario.

32. Configuración, balance y entornos

32.1. Settings assets

Los settings observados cubren checkout, customers, day cycle, economy, persistence, shopping, UI/UX y vertical slice. Cada asset debe tener defaults seguros y validación.

32.2. Separación de configuración

Se distinguen:

- configuración de diseño: capacidades, tiempos, precios y políticas;
- configuración técnica: rutas, flags, límites de logs y composición;
- preferencias del usuario: idioma, audio, accesibilidad y controles;
- estado de partida: dinero, inventario, día y progreso.

No deben almacenarse en el mismo objeto ni persistirse con el mismo ciclo de vida.

32.3. Defaults

Los defaults sirven para crear una partida o fixture, pero no deben ocultar referencias obligatorias ausentes. Una escena de producción con catálogo nulo debe fallar, no generar un catálogo silencioso distinto.

32.4. Configuración por entorno

Si se introducen perfiles Development, QA o Release, las diferencias deben ser explícitas:

- logging;
- scenario runners;
- cheats/debug;

- contenido de prueba;
- endpoints futuros;
- símbolos de compilación.

Una build Release no debe depender de assets presentes solo en carpetas Editor.

32.5. Balance

Los valores de balance deben vivir en datos cuando cambien con frecuencia. Sin embargo, una regla estructural no se convierte en un número configurable para evitar una decisión de diseño.

Cambios de balance requieren:

- rango válido;
- impacto esperado;
- test de límites;
- compatibilidad de save;
- registro de versión cuando afecte a partidas existentes.

33. Matriz técnica de propiedad de estado

Estado	Autoridad	Presentación	Persistencia	Mutación permitida
Grid/ocupación	PlacementOccupancyRegistry y records	views/ghosts	snapshot de placement	Casos de uso de placement
Productos	definiciones/registry	prefabs y UI	IDs, no assets	Authoring; no runtime salvo configuración aprobada
Inventario	InventoryContainer	displays, almacén, UI	records de inventario	servicios de transferencia/recepción/checkout
Pedidos	PurchaseOrder	panel proveedor	order records	SupplierOrderService y transiciones
Entregas	Delivery	cajas/receiving UI	delivery records	delivery/receiving services
Displays	DisplayInstance	fixture y productos visuales	display records	assignment/restock services
Clientes	CustomerInstance + session	agente y feedback	customer/session records cuando corresponda	services de spawn/transition/shopping
Reservas	registry de shopping	iconos/carrito	reservation records	reservation/checkout/cancellation services
Cola	CheckoutQueue	fila de agentes/UI	queue records	queue service/coordinator
Estación	CheckoutStation	mostrador/operador	station record	checkout/closure
Economía	EconomyLedger	HUD/resultados	ledger records	servicios económicos
Día	StoreDay	reloj/HUD	day record	day service/closure coordinator
Sesión	GameSession	menú/slot	session snapshot	game session service

Estado	Autoridad	Presentación	Persistencia	Mutación permitida
Preferencias	settings services	menús	repositorios globales	UI/settings service
Escena	context serializado	GameObjects	no como estado de negocio	Editor/composition

34. Contratos de restauración de estado

34.1. Principio

Cargar no consiste en deserializar y asignar campos arbitrariamente. Es una reconstrucción validada del estado autoritativo y su representación.

34.2. Fases

```
leer slot
→ validar archivo y schema
→ decodificar records
→ validar referencias internas
→ resolver contenido estático
→ construir agregados en memoria
→ validar invariantes globales
→ preparar escena/contexto
→ registrar objetos autorados
→ aplicar estado dinámico
→ enlazar vistas
→ ejecutar comprobación post-restore
→ habilitar input y simulación
```

34.3. Restauración transaccional

Si una fase falla antes de habilitar gameplay:

- se descarta el estado parcial;
- se conserva la sesión anterior o se vuelve al menú;
- se libera cualquier objeto temporal;
- se presenta la causa;
- no se sobrescribe el save;

- puede intentarse backup cuando corresponda.

34.4. Reconciliación de escena

Para cada entidad persistida se decide:

- ya existe como objeto autorado y se actualiza;
- es dinámica y se instancia;
- fue eliminada y no debe reaparecer;
- referencia contenido no disponible y requiere migración/fallo;
- es transitoria y no se restaura.

34.5. Postcondiciones

Antes de declarar carga completada:

- todos los IDs únicos;
- inventario consistente;
- ocupación consistente;
- anchors accesibles;
- reservas válidas;
- cola/estación coherentes;
- ledger válido;
- día válido;
- vistas registradas;
- no hay duplicados de roots;
- input en contexto correcto.

35. Checklist de implementación por subsistema

35.1. Nuevo tipo de dominio

- necesidad demostrada;

- namespace correcto;
- constructor validado;
- igualdad definida cuando procede;
- sin Unity;
- tests de valores límite;
- serialización evaluada;
- nombre coherente con lenguaje ubicuo.

35.2. Nuevo caso de uso

- comando o parámetros claros;
- precondiciones;
- causas de fallo;
- preflight;
- commit;
- idempotencia evaluada;
- eventos posteriores;
- tests de éxito/fallo/no mutación;
- proyección/UI actualizada.

35.3. Nuevo MonoBehaviour

- responsabilidad visual o de adapter;
- referencias serializadas explícitas;
- validación en `Awake` /Editor según necesidad;
- suscripciones liberadas;
- sin lógica de negocio duplicada;
- comportamiento con objeto deshabilitado/destruido;
- PlayMode test cuando existe riesgo.

35.4. Nuevo ScriptableObject

- ID estable si es contenido;

- defaults seguros;
- `OnValidate` o validador;
- conversión a modelo;
- catálogo y duplicados;
- impacto de save;
- fixture de test.

35.5. Cambio de escena

- jerarquía y contextos;
- GUIDs conservados;
- ausencia de duplicados;
- input/EventSystem;
- cámaras;
- lighting;
- colliders/layers;
- smoke test;
- Golden Path si afecta tienda;
- build list revisada.

35.6. Cambio de persistencia

- schema;
- records;
- codec;
- repository;
- migración;
- backup;
- fixtures antiguos;
- round trip;
- corrupción;
- UI de error;
- documentación.

35.7. Cambio de build o paquete

- ADR;
- versión congelada;
- compilación;
- tests;
- escenas;
- build externa;
- log;
- rollback.

Anexo A. Assembly map observado

Assembly	Ruta	Referencias	Plataforma	Auto referenc ed
VRMGames.CartridgeAndCloud.Editor.ProjectOrganization	Editor/ProjectOrganization/VRMGames.CartridgeAndCloud.Editor.ProjectOrganization.asmdef	VRMGames.CartridgeAndCloud.Domain , VRMGames.CartridgeAndCloud.Infrastructure , VRMGames.CartridgeAndCloud.Presentation	Editor	false
VRMGames.CartridgeAndCloud.Editor	Editor/VRMGames.CartridgeAndCloud.Editor.asmdef	VRMGames.CartridgeAndCloud.Application , VRMGames.CartridgeAndCloud.Presentation , VRMGames.CartridgeAndCloud.Infrastructure.InputSystem , Unity.InputSystem	Editor	false
VRMGames.CartridgeAndCloud.Application	Scripts/Application/VRMGames.CartridgeAndCloud.Application.asmdef	VRMGames.CartridgeAndCloud.Domain	Runtime/Editor según referencia	false
VRMGames.CartridgeAndCloud.Domain	Scripts/Domain/VRMGames.CartridgeAndCloud.Domain.asmdef	Ninguna	Runtime/Editor según referencia	false
VRMGames.CartridgeAndCloud.Infrastructure.InputSystem	Scripts/Infrastructure/InputSystem/VRMGames.CartridgeAndCloud.Infrastructure.InputSystem.asmdef	VRMGames.CartridgeAndCloud.Application , VRMGames.CartridgeAndCloud.Infrastructure , VRMGames.CartridgeAndCloud.Presentation , Unity.InputSystem	Runtime/Editor según referencia	false
VRMGames.CartridgeAndCloud.Infrastructure	Scripts/Infrastructure/VRMGames.CartridgeAndCloud.Infrastructure.asmdef	VRMGames.CartridgeAndCloud.Domain , VRMGames.CartridgeAndCloud.Application	Runtime/Editor según referencia	false
VRMGames.CartridgeAndCloud.Presentation	Scripts/Presentation/VRMGames.CartridgeAndCloud.Presentation.asmdef	VRMGames.CartridgeAndCloud.Domain , VRMGames.CartridgeAndCloud.Application	Runtime/Editor según referencia	false
VRMGames.CartridgeAndCloud.Runtime.VerticalSlicePhase1	Scripts/Runtime/VerticalSlicePhase1/VRMGames.CartridgeAndCloud.Runtime.VerticalSlicePhase1.asmdef	VRMGames.CartridgeAndCloud.Domain , VRMGames.CartridgeAndCloud.Application , VRMGames.CartridgeAndCloud.Infrastructure , VRMGames.CartridgeAndCloud.Presentation	Runtime/Editor según referencia	true
VRMGames.CartridgeAndCloud.Tests.EditMode	Tests/EditMode/VRMGames.CartridgeAndCloud.Tests.EditMode.asmdef	VRMGames.CartridgeAndCloud.Domain , VRMGames.CartridgeAndCloud.Application , VRMGames.CartridgeAndCloud.Infrastructure , VRMGames.CartridgeAndCloud.Presentation , UnityEngine.TestRunner , UnityEditor.TestRunner	Editor	false
VRMGames.CartridgeAndCloud.InputSystem.Tests.EditMode	Tests/InputSystem/EditMode/VRMGames.CartridgeAndCloud.InputSystem.Tests.EditMode.asmdef	VRMGames.CartridgeAndCloud.Application , VRMGames.CartridgeAndCloud.Infrastructure.InputSystem , Unity.InputSystem , UnityEngine.TestRunner , UnityEditor.TestRunner	Editor	false
VRMGames.CartridgeAndCloud.InputSystem.Tests.PlayMode	Tests/InputSystem/PlayMode/VRMGames.CartridgeAndCloud.InputSystem.Tests.PlayMode.asmdef	VRMGames.CartridgeAndCloud.Application , VRMGames.CartridgeAndCloud.Infrastructure , VRMGames.CartridgeAndCloud.Infrastructure.InputSystem , VRMGames.CartridgeAndCloud.Presentation , UnityEngine.TestRunner	Runtime/Editor según referencia	false
VRMGames.CartridgeAndCloud.Tests.PlayMode	Tests/PlayMode/VRMGames.CartridgeAndCloud.Tests.PlayMode.asmdef	VRMGames.CartridgeAndCloud.Domain , VRMGames.CartridgeAndCloud.Application , VRMGames.CartridgeAndCloud.Infrastructure , VRMGames.CartridgeAndCloud.Presentation , VRMGames.CartridgeAndCloud.Runtime.VerticalSlicePhase1 , UnityEngine.TestRunner	Runtime/Editor según referencia	false

Anexo B. Inventario de clases técnicas clave

La tabla no enumera todo el código. Señala puntos de entrada y contratos relevantes de la instantánea suministrada.

Subsistema	Archivo	Estado
Composición y escenas	Assets/_Project/Scripts/Infrastructure/SceneFlow/ApplicationRoot.cs	Presente
	Assets/_Project/Scripts/Application/SceneFlow/SceneTransitionGate.cs	Presente
	Assets/_Project/Scripts/Presentation/SceneFlow/MainMenuController.cs	Presente
	Assets/_Project/Scripts/Presentation/SceneFlow/StoreSceneController.cs	Presente
Sesión y guardado inicial	Assets/_Project/Scripts/Domain/GameSession/GameSession.cs	Presente
	Assets/_Project/Scripts/Application/GameSession/GameSessionService.cs	Presente
	Assets/_Project/Scripts/Infrastructure/GameSession/JsonSaveGameRepository.cs	Presente
Persistencia integrada	Assets/_Project/Scripts/Domain/Persistence/IntegratedGameStateSnapshot.cs	Presente
	Assets/_Project/Scripts/Domain/Persistence/SaveRecords.cs	Presente
	Assets/_Project/Scripts/Application/Persistence/IntegratedGameStateBuilder.cs	Presente
	Assets/_Project/Scripts/Application/Persistence/IntegratedSaveService.cs	Presente
	Assets/_Project/Scripts/Infrastructure/Persistence/IntegratedSaveJsonCodec.cs	Presente
	Assets/_Project/Scripts/Infrastructure/Persistence/JsonIntegratedSaveRepository.cs	Presente
Input	Assets/_Project/Scripts/Application/InputContexts/InputContextService.cs	Presente
	Assets/_Project/Scripts/Infrastructure/InputSystem/Actions/InputActionContextRouter.cs	Presente
	Assets/_Project/Scripts/Infrastructure/InputSystem/Actions/ProjectInputActionRuntimeBootstrap.cs	Presente
	Assets/_Project/Scripts/Infrastructure/InputSystem/Placement/PlacementInputActionDriver.cs	Presente
Movimiento y cámara	Assets/_Project/Scripts/Application/PlayerMovement/PlanarMovementCalculator.cs	Presente
	Assets/_Project/Scripts/Presentation/PlayerMovement/ClickToMoveAgent.cs	Presente
	Assets/_Project/Scripts/Application/Camera/OrbitCameraCalculator.cs	Presente
	Assets/_Project/Scripts/Presentation/Camera/OrbitCameraRig.cs	Presente
Grid, placement y acceso	Assets/_Project/Scripts/Domain/Grid/GridCoordinate.cs	Presente
	Assets/_Project/Scripts/Domain/Grid/GridFootprint.cs	Presente

Subsistema	Archivo	Estado
	Assets/_Project/Scripts/Application/Placement/PlacementOccupancyRegistry.cs	Presente
	Assets/_Project/Scripts/Application/Placement/PlacementPreviewCalculator.cs	Presente
	Assets/_Project/Scripts/Application/Access/GridAccessValidator.cs	Presente
	Assets/_Project/Scripts/Presentation/Placement/PlacementRuntimeController.cs	Presente
Productos e inventario	Assets/_Project/Scripts/Domain/Products/ProductDefinition.cs	Presente
	Assets/_Project/Scripts/Domain/Inventory/InventoryContainer.cs	Presente
	Assets/_Project/Scripts/Domain/Inventory/InventoryTransferService.cs	Presente
	Assets/_Project/Scripts/Infrastructure/Products/ProductCatalogAsset.cs	Presente
Pedidos y recepción	Assets/_Project/Scripts/Domain/Orders/PurchaseOrder.cs	Presente
	Assets/_Project/Scripts/Application/Orders/SupplierOrderService.cs	Presente
	Assets/_Project/Scripts/Domain/Receiving/Delivery.cs	Presente
	Assets/_Project/Scripts/Application/Receiving/ReceivingService.cs	Presente
Displays y reposición	Assets/_Project/Scripts/Domain/Displays/DisplayInstance.cs	Presente
	Assets/_Project/Scripts/Application/Displays/DisplayRestockService.cs	Presente
	Assets/_Project/Scripts/Infrastructure/Displays/DisplayRuntimeAuthoring.cs	Presente
Clientes y shopping	Assets/_Project/Scripts/Domain/Customers/CustomerInstance.cs	Presente
	Assets/_Project/Scripts/Application/Customers/CustomerSpawnService.cs	Presente
	Assets/_Project/Scripts/Domain/Shopping/ShoppingReservations.cs	Presente
	Assets/_Project/Scripts/Application/Shopping/ShoppingReservationService.cs	Presente
	Assets/_Project/Scripts/Application/Shopping/ShoppingIntentAndFlowServices.cs	Presente
Checkout	Assets/_Project/Scripts/Domain/Checkout/CheckoutQueue.cs	Presente
	Assets/_Project/Scripts/Domain/Checkout/CheckoutStation.cs	Presente
	Assets/_Project/Scripts/Application/Checkout/CheckoutService.cs	Presente
	Assets/_Project/Scripts/Application/Checkout/CheckoutCancellationService.cs	Presente
Día y economía	Assets/_Project/Scripts/Domain/DayCycle/StoreDayCore.cs	Presente
	Assets/_Project/Scripts/Application/DayCycle/StoreClosureCoordinator.cs	Presente
	Assets/_Project/Scripts/Domain/Economy/EconomyLedger.cs	Presente

Subsistema	Archivo	Estado
	Assets/_Project/Scripts/Application/Economy/DailyResultsService.cs	Presente
UI, tutorial y accesibilidad	Assets/_Project/Scripts/Application/UIUX/StoreUiProjectionService.cs	Presente
	Assets/_Project/Scripts/Application/UIUX/TutorialService.cs	Presente
	Assets/_Project/Scripts/Infrastructure/UIUX/Sprint15RuntimeCompositionRoot.cs	Presente
	Assets/_Project/Scripts/Infrastructure/UIUX/StoreHudScreen.cs	Presente
Vertical slice representativo	Assets/_Project/Scripts/Runtime/VerticalSlicePhase1/Sprint16Phase1RuntimeRoot.cs	Presente
	Assets/_Project/Scripts/Runtime/VerticalSlicePhase1/Phase1PlacementCompatibilityBridge.cs	Presente
	Assets/_Project/Scripts/Runtime/VerticalSlicePhase1/RepresentativeStoreVisualBuilder.cs	Presente
	Assets/_Project/Scripts/Infrastructure/VerticalSlicePhase1/Phase1RuntimeAssetRegistryAsset.cs	Presente

Anexo C. Distribución de código

Área	Archivos C#
Scripts/Domain	109
Scripts/Application	87
Scripts/Infrastructure	79
Scripts/Presentation	12
Scripts/Runtime	13
Editor	14
Tests/EditMode	128
Tests/PlayMode	11
Tests/InputSystem	4

Anexo D. Escenas y build

D.1. Escenas presentes

Escena	Build actual
Assets/_Project/Scenes/Bootstrap.unity	Incluida
Assets/_Project/Scenes/MainMenu.unity	Incluida
Assets/_Project/Scenes/Store.unity	Incluida
Assets/_Project/Scenes/StoreInitial.unity	No incluida
Assets/_Project/Scenes/TestLab.unity	Incluida

D.2. Orden observado

```
0 Assets/_Project/Scenes/Bootstrap.unity
1 Assets/_Project/Scenes/MainMenu.unity
2 Assets/_Project/Scenes/Store.unity
3 Assets/_Project/Scenes/TestLab.unity
```

StoreInitial.unity deberá añadirse únicamente cuando el gate lo autorice.

Anexo E. Trazabilidad de fuentes

Fuente	Líneas	Palabras	SHA-256
TDD v0.3	582	2532	86160690276d137118b119d6376eee8d1834097dbfe92fb6da7d18356d737eb2
TDD v0.4	623	2793	bd2237fa0323821c4d4717d89940f005bbce2c2232ac91a4dd72f54301fb879d
TDD v0.5	171	533	2067d16cac0aae694c69895371dca234c4daaa5258f13922844214283bd09e25
TDD v0.6	118	486	daa5578e670e61ffe40db754ef512d02885d1bb7600c74487194ad8ec27cb2ce
00_Enfoque_y_Alcance.md	4495	18925	63dd6f0f1b9c2c80be2aec55718d18d9d031ce4acb14f677769d6e69ceaad21f
01_Game_Design_Document.md	4490	19760	a9cd9e3203abc5269c25c59dc7f626b0771df4ec2d65e781109ff146bceb2177

Fuente	Líneas	Palabras	SHA-256
02_Vertical_Slice_Specification.md	1505	9834	75893f130116e88a08b8da5590dd81876a234581081eafaa8d08374c1a60513f

E.1. Resoluciones principales

- TDD v0.6 prevalece para autoría de escena y migración representativa.
- TDD v0.5 prevalece para la fundación implementada de grid, acceso e input.
- TDD v0.4 aporta la arquitectura extensa por capas y contratos futuros.
- TDD v0.3 se conserva como contexto histórico.
- La instantánea de código confirma el estado observado y revela deuda transitoria.
- 00_Enfoque_y_Alcance.md, GDD y Vertical Slice Specification limitan cualquier interpretación expansiva del TDD histórico.

Anexo F. Glosario técnico

Término	Definición
Adapter	Implementación técnica de un puerto definido por una capa interior.
Aggregate	Conjunto de objetos de dominio que preserva invariantes como unidad.
Anchor	Punto funcional requerido para acceso, interacción o composición.
Atomicidad	La operación se aplica completa o no deja cambios.
Authoring	Configuración explícita de escenas y assets en Editor.
Composition root	Lugar donde se construyen servicios y se conectan dependencias.
Context	Frontera funcional o componente que reúne referencias de una escena.
DTO/record	Forma serializable sin comportamiento de negocio.
Fallback	Ruta temporal segura conservada durante una migración.
Golden Path	Recorrido integrado obligatorio del vertical slice.
Idempotencia	Repetir una operación identificada no duplica su efecto.
Invariante	Condición que siempre debe cumplirse en un estado válido.
Ledger	Libro mayor de movimientos monetarios.
Mutation	Cambio de estado autoritativo.

Término	Definición
Port	Interfaz de una capa interior que Infrastructure implementa.
Preflight	Validación completa previa al commit.
Projection	Modelo de lectura preparado para UI.
Registry	Colección validada que resuelve IDs.
Snapshot	Representación coherente y persistente del estado.
Stable ID	Identidad independiente de nombres, vistas y sesiones Unity.
Value object	Tipo definido por su valor y validación, sin identidad propia.

Estado del documento: fuente vigente de diseño técnico para la nueva carpeta `Documentacion/`. Debe guardarse como `Documentacion/03_Technical_Design_Document.md`.

Fin del documento.
Fuente: 03_Technical_Design_Document.md · SHA-256: `66264703c5ff4959d03093690e9845a98ec0e5025e685b9a639ac8cbb616cd12`
Diseño PDF: paleta VRM Games definida en la documentación de Cartridge & Cloud.